

Cvičné úlohy ke státnicím

Úroveň 1 (cíl: > 90 % pravděpodobnost složení)

Informatika - Umělá inteligence, zaměření Strojové učení (UI-SU) | MFF UK

Jak je to zkalibrované (přečti první)

Zkouška pro UI-SU = 8 otázek (2 spol. matematika, 2 spol. informatika, 4 specializace), každá 0-3 body. Složení není hra o celkový součet, ale o **dvě nezávislé podlahy**:

$$\text{projdeš} \iff \underbrace{(\text{společné} \geq 5/12)}_{\text{shovívavé: } 2+1+1+1} \text{ a } \underbrace{(\text{specializace} \geq 7/12)}_{\text{přísné: } 2+2+2+1}.$$

Klíčový důsledek: ve specializaci musíš dostat **tři ze čtyř** otázek na ≥ 2 body, kdežto společné okruhy ustojíš jednou jistotou plus definicemi. Proto úroveň 1 obsahuje:

- plně zvládnutí nejčastějších témat (regrese, shlukování, evaluační metriky, rozhodovací stromy; lineární algebra, automaty);
- u *každé* rodiny specializace (i „Základy UI“) aspoň **2-bodovou zálohu** = přesná definice + jeden kanonický postup;
- u společné informatiky „**nikdy nedostat 0**“ i na implementační otázce (binární soubor, semafor, OO v C#);
- přesné definice/znění vět napříč okruhy (nejlevnější body).

Jak procvičovat: u každé úlohy zakryj *Řešení*, odpověz nahlas a miř na „2 body“ (definice + jeden postup). Zelené pole *Jádro na 2 body* u specializace je to, co musíš umět nazpaměť (cílí na to, abys nikdy nedostal 0-1 z pokryté otázky). Oranžová *Kalibrace* říká, proč úloha sytí kterou podlahu.

Proč > 90 % (a poctivě): vázající je podlaha specializace; modelově $P(\text{spec} \geq 7) \approx 0,95$ (tři ze čtyř otázek na ≥ 2 body) a $P(\text{spol.} \geq 5) \approx 0,96$, tedy $P(\text{složím}) \approx 0,92$. Dvě věci, které tuto hodnotu zvedají a jsou už zapracované: pokrytí *všech* rodin „Základy UI“ (i plánování, situační kalkulus, mechanism design, POMDP), aby vždy přítomný AI slot nepadl na nepokryté téma, a *Jádro na 2 body* snižující šanci na skóre 0-1 u ML otázek.

Aditivní struktura: sešity se sčítají - úroveň 1 (tento) $\rightarrow >90\%$; + úroveň 2 (výpočetní varianty, důkazové náčrty) $\rightarrow >95\%$; + úroveň 3 (vzácný „ocas“ požadavku, zkoušky nanečisto) $\rightarrow \sim 97,5\%$. Každý vyšší sešit obsahuje jen *nové* úlohy, takže se nic neopakuje. Z 9 zkoušek dat a velkého oficiálního okruhu nelze poctivě garantovat > 99%; reálný strop pilného studenta je $\sim 97-98,5\%$.

Obsah

1 Společná matematika	1
2 Společná informatika	7
3 Specializace: Strojové učení (Téma 3)	15
4 Specializace: Základy umělé inteligence (Téma 1)	22

1 Společná matematika

Úloha 1. společná matematika Vlastní čísla, diagonalizace a symetrické matice

Je dána reálná matice

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

- (a) **1 b** Definujte vlastní číslo a vlastní vektor matice a charakteristický polynom. Uveďte vztah mezi vlastními čísly a determinantem (resp. regularitou) matice.

- (b) **2b** Spočítejte vlastní čísla matice A a ke každému najděte vlastní vektor.
- (c) **3b** Rozhodněte, zda je A diagonalizovatelná. Pokud ano, napište matice P, D tak, že $A = PDP^{-1}$. Vysvětlete, proč je každá reálná symetrická matice (ortogonálně) diagonalizovatelná.

(a) **Definice.** Číslo $\lambda \in \mathbb{R}$ (obecně $\mathbb{C}$) je *vlastní číslo* matice A , pokud existuje nenulový vektor v s $Av = \lambda v$; takový v je *vlastní vektor*. Vlastní čísla jsou kořeny *charakteristického polynomu* $p_A(\lambda) = \det(A - \lambda I)$. Platí $\det A = \prod_i \lambda_i$ (součin vlastních čísel s násobností), takže A je regulární právě tehdy, když 0 není vlastní číslo.

(b) **Výpočet.**

$$\det(A - \lambda I) = \det \begin{pmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{pmatrix} = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3).$$

Vlastní čísla: $\lambda_1 = 3$, $\lambda_2 = 1$.

- $\lambda_1 = 3$: $(A - 3I) = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix}$, řešení $v_1 = (1, 1)^\top$.
- $\lambda_2 = 1$: $(A - I) = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, řešení $v_2 = (1, -1)^\top$.

(c) **Diagonalizace.** A má dvě různá vlastní čísla, příslušné vlastní vektory jsou nezávislé, takže A je diagonalizovatelná:

$$P = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad D = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}, \quad A = PDP^{-1}.$$

A je navíc symetrická. *Spektrální věta*: každá reálná symetrická matice má reálná vlastní čísla a vlastní vektory příslušné různým vlastním číslům jsou na sebe kolmé, takže je lze znormovat na ortonormální bázi. Pak $A = QDQ^\top$ s ortogonální Q ($Q^{-1} = Q^\top$). Zde $v_1 \cdot v_2 = 1 \cdot 1 + 1 \cdot (-1) = 0$, takže $Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$.

Kalibrace: Lineární algebra je nejčastější okruh společné matematiky (65 % zkoušek). Část (a)+(b) = 2 body bezpečně sytí podlahu společných okruhů (cíl $\geq 5/12$). Definice z (a) je nejlevnější bod, nikdy ji nevynechej.

Pozor (častá chyba): U diagonalizovatelnosti se neptáme jen „jdou spočítat vlastní čísla“, ale zda existuje báze z vlastních vektorů (tj. zda je algebraická = geometrická násobnost). Různá vlastní čísla to zaručí; u násobných je nutno ověřit dimenzi vlastního podprostoru.

Úloha 2. **společná matematika** **Soustavy rovnic, determinant, hodnost, skalární součin a ortogonalizace**

- (a) **1b** Definujte hodnost matice, regulární a inverzní matici a determinant. Uveďte vztah mezi regularitou, determinantem a vlastními čísly.
- (b) **2b** Spočítejte determinant matice $B = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 1 & 0 & 1 \end{pmatrix}$ a rozhodněte o její regularitě. Popište, jak Gaussova eliminace určí množinu řešení soustavy.
- (c) **3b** Definujte skalární součin a kolmost. Gram-Schmidtovou ortogonalizací zpracujte vektory $v_1 = (1, 1, 0)$, $v_2 = (1, 0, 1)$ a výsledek znormujte.

(a) *Hodnost* rank A je dimenze řádkového (= sloupcového) prostoru, tj. počet nenulových řádků v odstupňovaném tvaru. Matice $A \in \mathbb{R}^{n \times n}$ je *regulární*, má-li hodnost n ; pak existuje *inverzní* A^{-1} s $AA^{-1} = I$. *Determinant* $\det A$ je multilineární antisymetrická funkce sloupce s $\det I = 1$; platí $\det(AB) = \det A \det B$, $\det A^T = \det A$. Ekvivalentně: A regulární $\iff \det A \neq 0 \iff 0$ není vlastní číslo $\iff \text{rank } A = n$.

(b) Laplaceuv rozvoj podle prvního řádku:

$$\det B = 1 \cdot \det \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix} - 2 \cdot \det \begin{pmatrix} 0 & 3 \\ 1 & 1 \end{pmatrix} + 0 = 1 \cdot 1 - 2 \cdot (0 - 3) = 1 + 6 = 7 \neq 0,$$

takže B je regulární. *Gaussova eliminace*: soustavu $Ax = b$ převedeme řádkovými úpravami na odstupňovaný tvar. Pak: pokud vznikne řádek $0 = c$ s $c \neq 0$, soustava nemá řešení; jinak je počet volných proměnných = (počet neznámých) – rank A a řešení je jednoznačné právě tehdy, když rank A = počtu neznámých.

(c) *Skalární součin* na reálném vektorovém prostoru V je zobrazení $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{R}$, které je *symetrické* ($\langle x, y \rangle = \langle y, x \rangle$), *bilinéární* (lineární v každé složce) a *pozitivně definitní* ($\langle x, x \rangle \geq 0$, s rovností jen pro $x = 0$). (V komplexním případě se požaduje hermitovskost/sesquilinearita.) Standardní příklad v $\mathbb{R}^n$ je $\langle x, y \rangle = \sum_i x_i y_i$. Indukovaná *norma* $\|x\| = \sqrt{\langle x, x \rangle}$; vektory jsou *kolmé*, je-li $\langle x, y \rangle = 0$. Gram-Schmidt:

$$u_1 = v_1 = (1, 1, 0), \quad u_2 = v_2 - \frac{\langle v_2, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1 = (1, 0, 1) - \frac{1}{2}(1, 1, 0) = \left(\frac{1}{2}, -\frac{1}{2}, 1\right).$$

Kontrola: $\langle u_1, u_2 \rangle = \frac{1}{2} - \frac{1}{2} + 0 = 0$. Znormování: $e_1 = \frac{1}{\sqrt{2}}(1, 1, 0)$, $\|u_2\| = \sqrt{\frac{1}{4} + \frac{1}{4} + 1} = \sqrt{3/2}$, takže $e_2 = \frac{1}{\sqrt{6}}(1, -1, 2)$.

Kalibrace: Lineární algebra je nejčastější okruh společné matematiky; tato úloha jistí „výpočetní“ polovinu (soustavy/determinant/ortogonalizace) vedle úlohy o vlastních číslech. Definice $z(a) = \text{jistý } 1 \text{ bod}$.

Pozor (častá chyba): Determinant počítej rozvojem podle řádku/sloupce s nejvíce nulami. U Gram-Schmidta nezapomeň odečítat *projekci* (s dělením $\langle u_1, u_1 \rangle$), ne jen vektor.

Úloha 3. **společná matematika** **Limity, spojitost, derivace a Taylorův polynom**

- (a) **1 b** Definujte limitu funkce v bodě a spojitost funkce. Zformulujte větu o nabývání mezihodnot a větu o nabývání maxima na uzavřeném intervalu.
- (b) **2 b** Spočtěte $\lim_{x \rightarrow 0} \frac{\sin x - x}{x^3}$. Pro $f(x) = x^3 - 3x$ najděte lokální extrémy a intervaly monotonie.
- (c) **3 b** Napište Taylorův polynom (limitní formu) a pomocí něj zdůvodněte výsledek limity z (b). Určete konvexitu/konkavitu f .

(a) $\lim_{x \rightarrow a} f(x) = L$, pokud $\forall \varepsilon > 0 \exists \delta > 0 : 0 < |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon$. Funkce je *spojitá* v a , je-li $\lim_{x \rightarrow a} f(x) = f(a)$. *Věta o mezihodnotě (Bolzano)*: je-li f spojitá na $[a, b]$ a y leží mezi $f(a), f(b)$, existuje $c \in [a, b]$ s $f(c) = y$. *Weierstrass*: spojitá funkce na uzavřeném omezeném intervalu nabývá maxima i minima.

(b) Limita je typu $0/0$. l'Hospital třikrát, nebo lépe Taylor: $\sin x = x - \frac{x^3}{6} + o(x^3)$, takže

$$\frac{\sin x - x}{x^3} = \frac{-x^3/6 + o(x^3)}{x^3} \rightarrow -\frac{1}{6}.$$

Pro $f(x) = x^3 - 3x$: $f'(x) = 3x^2 - 3 = 3(x - 1)(x + 1)$, nulové body $x = \pm 1$. $f' > 0$ na

$(-\infty, -1) \cup (1, \infty)$ (rostoucí), $f' < 0$ na $(-1, 1)$ (klesající). V $x = -1$ lokální maximum, v $x = 1$ lokální minimum.

(c) *Tayloruv polynom řádu n v bodě a :* $T_n(x) = \sum_{k=0}^n \frac{f^{(k)}(a)}{k!} (x-a)^k$, přičemž $f(x) = T_n(x) + o((x-a)^n)$ (limitní/Peanova forma). Rozvoj $\sin x$ kolem 0 dává přímo čísel $-x^3/6 + o(x^3)$, odkud limita $-1/6$. Konvexita: $f''(x) = 6x$, takže f je konkávní na $(-\infty, 0)$ a konvexní na $(0, \infty)$; v $x = 0$ je inflexní bod.

Kalibrace: Analýza je druhý nejčastější okruh matematiky. Definice limity/spojitosti + jeden spočtený limit/extrem spolehlivě dají 2 body do podlahy společných okruhů.

Pozor (častá chyba): l'Hospital lze použít jen na typy $\frac{0}{0}$ nebo $\frac{\infty}{\infty}$ a po ověření, že limita podílu derivací existuje. Taylor je často rychlejší a méně chybový.

Úloha 4. **společná matematika** Integrály, primitivní funkce a řady

- (a) **1 b** Definujte primitivní funkci a Riemannuv integrál a uveďte jejich vztah (Newtonova-Leibnizova formule). Definujte součet geometrické řady a uveďte vzorec pro $\sum_{n=0}^{\infty} ar^n$.
- (b) **2 b** Spočítejte $\int xe^x dx$ metodou per partes a určete $\int_0^1 xe^x dx$.
- (c) **3 b** Napište vzorec pro obsah pod grafem a pro objem rotačního tělesa. Vysvětlete odhad součtu řady integrálem (integrální kritérium).

(a) F je *primitivní funkce* k f na intervalu, je-li $F' = f$. *Riemannuv integrál* $\int_a^b f$ je společná limita horních a dolních součtu přes zjemňující se dělení (existuje, je-li f např. spojitá). *Newton-Leibniz:* je-li F primitivní k spojitě f , pak $\int_a^b f = F(b) - F(a)$. *Geometrická řada:* $\sum_{n=0}^{\infty} ar^n = \frac{a}{1-r}$ pro $|r| < 1$ (jinak diverguje); částečný součet $\sum_{n=0}^N ar^n = a \frac{1-r^{N+1}}{1-r}$.

(b) Per partes $u = x$, $dv = e^x dx$, tedy $du = dx$, $v = e^x$:

$$\int xe^x dx = xe^x - \int e^x dx = (x-1)e^x + C.$$

Určitě: $\int_0^1 xe^x dx = [(x-1)e^x]_0^1 = (0) \cdot e - (-1) \cdot 1 = 1$.

(c) *Obsah* pod nezápornou f na $[a, b]$: $S = \int_a^b f(x) dx$. *Objem* tělesa vzniklého rotací grafu kolem osy x : $V = \pi \int_a^b f(x)^2 dx$. *Integrální kritérium:* je-li f kladná klesající, pak $\sum_{n=1}^{\infty} f(n)$ konverguje právě tehdy, když konverguje $\int_1^{\infty} f(x) dx$, a platí odhad $\int_1^{N+1} f \leq \sum_{n=1}^N f(n) \leq f(1) + \int_1^N f$.

Kalibrace: Integrál + geometrická řada jsou opakované matematické podotázky (geometrická řada byla i samostatná otázka). Vzorec per partes a součet geometrické řady umět zapaměti.

Pozor (častá chyba): Per partes: vol u tak, aby se derivováním zjednodušilo (polynom), dv tak, aby šlo integrovat. Geometrická řada konverguje jen pro $|r| < 1$ - vždy ověř.

Úloha 5. **společná matematika** Teorie grafu: souvislost, stromy, barevnost, rovinnost, toky

- (a) **1 b** Definujte graf, stupeň vrcholu, souvislost a komponentu, strom (uveďte aspoň dvě ekvivalentní charakteristiky) a bipartitní graf.
- (b) **2 b** Definujte dobré obarvení a barevnost $\chi(G)$. Uveďte vztah barevnosti a klikovosti. Určete $\chi(C_5)$ (kružnice délky 5) a zdůvodněte.
- (c) **3 b** Zformulujte Eulerovu formuli pro rovinný graf a odvoďte horní mez počtu hran. Definujte síť, tok a řez a uveďte větu o max. toku a min. řezu.

(a) Graf $G = (V, E)$, $E \subseteq \binom{V}{2}$. Stupeň $\deg(v)$ = počet hran u v (princip podání rukou: $\sum_v \deg v = 2|E|$). G je *souvislý*, existuje-li cesta mezi každou dvojicí vrcholu; *komponenta* = maximální souvislý podgraf. *Strom* = souvislý graf bez kružnice; ekvivalentně: souvislý s $|E| = |V| - 1$; nebo: mezi každými dvěma vrcholy existuje právě jedna cesta. *Bipartitní* graf: V lze rozdělit na dvě části tak, že každá hrana vede mezi nimi (ekvivalentně: neobsahuje lichou kružnici).

(b) *Dobré obarvení* k barvami: zobrazení $V \rightarrow \{1, \dots, k\}$ tak, že sousední vrcholy mají různou barvu. *Barevnost* $\chi(G)$ = nejmenší takové k . Platí $\chi(G) \geq \omega(G)$ (klikovost; kliku potřebuje tolik barev, kolik má vrcholu), ale rovnost obecně neplatí. C_5 : jako lichá kružnice není bipartitní, takže $\chi > 2$; třemi barvami obarvit lze, tedy $\chi(C_5) = 3$.

(c) *Eulerova formule*: pro souvislý rovinný graf s V vrcholy, E hranami a F stěnami platí $V - E + F = 2$. U jednoduchého grafu s $V \geq 3$ je každá stěna ohraničena ≥ 3 hranami a každá hrana sousedí se 2 stěnami, takže $2E \geq 3F$; dosazením $F = 2 - V + E$ dostaneme $E \leq 3V - 6$. (U grafu s mosty je třeba argument doplnit, závěr ale platí.) *Síť* = orientovaný graf s kapacitami c , zdrojem s a stokem t ; *tok* f splňuje $0 \leq f \leq c$ a Kirchhoffuv zákon (zachování v každém vnitřním vrcholu); *řez* = rozdělení vrcholu na $S \ni s$ a $T \ni t$, jeho kapacita = součet kapacit hran z S do T . *Věta*: velikost maximálního toku se rovná kapacitě minimálního řezu (max-flow = min-cut).

Kalibrace: Grafy jsou stabilní okruh matematiky (barevnost, eulerovské/rovinné grafy, souvislost se v zadáních opakují). Definice + jedno odvození (např. $E \leq 3V - 6$) = 2 body.

Pozor (častá chyba): $\chi \geq \omega$ je jen nerovnost - existují grafy bez trojúhelníku s libovolně velkou barevností. Eulerova formule platí pro *souvislé* rovinné grafy.

Úloha 6. společná matematika Pravděpodobnost a statistika: Bayes, střední hodnota, rozptyl, CLT, testování

- 1 b** Definujte pravděpodobnostní prostor, podmíněnou pravděpodobnost a nezávislost jevu. Zformulujte Bayesovu větu.
- 2 b** Definujte střední hodnotu a rozptyl náhodné veličiny. Uveďte linearitu střední hodnoty a vzorec pro rozptyl součtu. Spočítejte $\mathbb{E}X$ a $\text{var } X$ pro hod férovou kostkou.
- 3 b** Zformulujte zákon velkých čísel a centrální limitní větu. Popište testování hypotéz: hladina významnosti, chyba 1. a 2. druhu.

(a) *Pravděpodobnostní prostor* $(\Omega, \mathcal{F}, P)$: množina elementárních jevu Ω , σ -algebra jevu $\mathcal{F}$, míra P s $P(\Omega) = 1$. *Podmíněná* $P(A | B) = \frac{P(A \cap B)}{P(B)}$ pro $P(B) > 0$. Jevy *nezávislé*, je-li $P(A \cap B) = P(A)P(B)$. *Bayes*: $P(A | B) = \frac{P(B | A)P(A)}{P(B)}$, kde $P(B) = \sum_i P(B | A_i)P(A_i)$ (úplná pravděpodobnost).

(b) $\mathbb{E}X = \sum_x x P(X = x)$ (diskrétně), resp. $\mathbb{E}X = \int_{-\infty}^{\infty} x f_X(x) dx$ (spojitě). *Linearita*: $\mathbb{E}(aX + bY) = a\mathbb{E}X + b\mathbb{E}Y$ (vždy, i pro závislé). $\text{var } X = \mathbb{E}(X - \mathbb{E}X)^2 = \mathbb{E}X^2 - (\mathbb{E}X)^2$; $\text{var}(aX) = a^2 \text{var } X$. Obecně $\text{var}(X + Y) = \text{var } X + \text{var } Y + 2 \text{cov}(X, Y)$, takže pro *nezávislé* (kde $\text{cov} = 0$) je $\text{var}(X + Y) = \text{var } X + \text{var } Y$. Kostka: $\mathbb{E}X = \frac{1+2+\dots+6}{6} = 3,5$, $\mathbb{E}X^2 = \frac{1^2+2^2+\dots+6^2}{6} = \frac{91}{6}$, $\text{var } X = \frac{91}{6} - \left(\frac{49}{4}\right) = \frac{35}{12} \approx 2,92$.

(c) *ZVČ*: prumer $\bar{X}_n$ nezávislých stejně rozdělených veličin konverguje k $\mathbb{E}X$. *CLT*: pro nezávislé stejně rozdělené se střední hodnotou μ a rozptylem σ^2 platí $\frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma} \xrightarrow{d} \mathcal{N}(0, 1)$, tj. normovaný prumer má asymptoticky normální rozdělení. *Testování*: nulová hypotéza H_0 vs. alternativa; *hladina významnosti* α = max. přípustná pravděpodobnost chyby 1. druhu (zamítnout platné

H_0); chyba 2. druhu β = nezamítnout neplatné H_0 ; síla testu = $1 - \beta$.

Kalibrace: Pravděpodobnost a teorie informace se v matematice objevují středně často; definice Bayes/EX/var + jeden výpočet je bezpečná záloha. Tyto pojmy navíc podpírají ML specializaci (regrese, naivní pravděpodobnostní úvahy).

Pozor (častá chyba): Linearita střední hodnoty platí i pro závislé veličiny; aditivita rozptylu jen pro nezávislé (jinak +2 cov). Hladina významnosti řídí chybu 1. druhu, ne 2.

Úloha 7. **společná matematika** Logika: normální tvary, sémantika, rezoluce

- (a) **1 b** Vysvětlete základní pojmy syntaxe výrokové a predikátové logiky (formule, otevřená/uzavřená formule). Definujte CNF, DNF a PNF.
- (b) **2 b** Definujte model, splnitelnost, tautologii a důsledek. Převedte $(p \rightarrow q)$ na CNF a rozhodněte, zda je $(p \rightarrow q) \wedge p \rightarrow q$ tautologie.
- (c) **3 b** Popište rezoluci jako dokazovací metodu. Rezolucí ukažte, že množina klauzulí $\{p \vee q, \neg p, \neg q\}$ je nespílitelná.

(a) *Formule* se tvoří z prvovýroku (resp. atomických formulí s predikáty, termy a kvantifikátory) logickými spojkami. Ve výrokové logice není kvantifikace; v predikátové je formule *otevřená*, neobsahuje-li žádný kvantifikátor, a *uzavřená* (sentence), nemá-li žádnou volnou proměnnou (každý výskyt proměnné je vázán kvantifikátorem). *CNF* = konjunkce disjunkcí literálů (klauzulí); *DNF* = disjunkce konjunkcí literálů; *PNF* (prenexní) = všechny kvantifikátory na začátku, za nimi otevřené jádro.

(b) *Model* teorie/formule = ohodnocení (struktura), v němž je pravdivá. Formule je *splnitelná*, má-li model; *tautologie*, je-li pravdivá v každém ohodnocení; φ je *důsledek* teorie T ($T \models \varphi$), platí-li ve všech modelech T . Převod: $p \rightarrow q \equiv \neg p \vee q$ (už je CNF, jediná klauzule). Výraz $((p \rightarrow q) \wedge p) \rightarrow q$ je *modus ponens*; pravdivostní tabulkou (4 řádky) vyjde vždy 1, je to tautologie.

(c) *Rezoluce*: pracuje s klauzulemi (CNF); rezolventa dvojice klauzulí $C_1 \vee \ell$ a $C_2 \vee \neg \ell$ je $C_1 \vee C_2$. Množina je nespílitelná, lze-li z ní odvodit prázdnou klauzuli $\square$. Zde:

$$\{p \vee q\}, \{\neg p\} \Rightarrow \{q\}; \quad \{q\}, \{\neg q\} \Rightarrow \square.$$

Odvodili jsme prázdnou klauzuli, takže $\{p \vee q, \neg p, \neg q\}$ je nespílitelná. (Doplňkově: věty o kompaktnosti a úplnosti zaručují, že rezoluce je úplná dokazovací metoda - detail je nad rámec úrovně 1.)

Kalibrace: Logika je vzácnější matematický okruh, ale definice (CNF/DNF, model, splnitelnost) jsou levné body a rezoluce/DPLL se prolínají i se specializací „Základy UI“. Tady stačí 2-bodová záloha.

Pozor (častá chyba): Důsledek ($\models$) je sémantický pojem (přes modely), dokazatelnost ($\vdash$) syntaktický (přes odvození); věta o úplnosti je spojuje. Nezaměňuj splnitelnost (existuje model) s tautologií (všechny modely).

Úloha 8. **společná matematika** Diskrétní matematika: relace, kombinatorika, inkluze-exkluze

- (a) **1 b** Definujte vlastnosti binární relace (reflexivita, symetrie, antisymetrie, tranzitivita), ekvivalenci a rozkladové třídy a částečné uspořádání (řetězec, antiřetězec).
- (b) **2 b** Kolik je prostých a kolik všech funkcí z m -prvkové do n -prvkové množiny? Zformulujte binomickou větu a princip inkluze a exkluze.

- (c) **3 b** Pomocí inkluze-exkluze odvoďte počet surjekcí z m -prvkové na n -prvkovou množinu a počet permutací bez pevného bodu (problém šatnářky).

(a) Relace $R \subseteq X \times X$ je *reflexivní* ($\forall x : xRx$), *symetrická* ($xRy \Rightarrow yRx$), *antisymetrická* ($xRy \wedge yRx \Rightarrow x = y$), *tranzitivní* ($xRy \wedge yRz \Rightarrow xRz$). *Ekvivalence* = reflexivní + symetrická + tranzitivní; *třída* prvku x je $[x] = \{y \in X \mid xRy\}$. Třídy jsou neprázdné, po dvou disjunktní a pokrývají X (tvoří rozklad X). *Částečné uspořádání* = reflexivní + antisymetrické + tranzitivní; *řetězec* = po dvou porovnatelné prvky, *antiřetězec* = po dvou neporovnatelné.

(b) Všechny funkce $X \rightarrow Y$ je n^m ; *prostých* (injekcí) je pro $m \leq n$ právě $n(n-1) \cdots (n-m+1) = \frac{n!}{(n-m)!}$, pro $m > n$ je jich 0 (Dirichletův princip). *Binomická věta*: $(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$. *Inkluze-exkluze*: $|\bigcup_{i=1}^n A_i| = \sum_i |A_i| - \sum_{i < j} |A_i \cap A_j| + \cdots + (-1)^{n+1} |A_1 \cap \cdots \cap A_n|$.

(c) *Surjekce* z m na n : od všech n^m funkcí odečteme ty, které vynechají aspoň jeden prvek cíle. Označíme-li A_i funkce vynechávající i -tý prvek, dostaneme inkluzi-exkluzi

$$\#\text{surjekcí} = \sum_{i=0}^n (-1)^i \binom{n}{i} (n-i)^m.$$

Problém šatnářky (derangement): permutace $\{1, \dots, n\}$ bez pevného bodu; A_i = permutace s $\pi(i) = i$, $|A_{i_1} \cap \cdots \cap A_{i_k}| = (n-k)!$, takže

$$D_n = \sum_{k=0}^n (-1)^k \binom{n}{k} (n-k)! = n! \sum_{k=0}^n \frac{(-1)^k}{k!} \approx \frac{n!}{e}.$$

(Hallová věta o SRR a párování v bipartitním grafu je příbuzné téma - viz vyšší úroveň.)

Kalibrace: Diskrétní matematika dává levné definiční body (relace, ekvivalence) a jeden klasický I-E výpočet. Inkluze-exkluze je opakovaný typ.

Pozor (častá chyba): Antisymetrie není „opak symetrie“. Počet surjekcí není $\binom{n}{?}$ trik - jde přes inkluzi-exkluzi; nezapomeň člen $i = 0$ ($= n^m$).

2 Společná informatika

Úloha 9. **společná informatika** **Regulární jazyky: automaty, výrazy, uzávěry, neregularita**

- (a) **1 b** Definujte deterministický (DFA) a nedeterministický (NFA) konečný automat, regulární gramatiku a regulární výraz. Zformulujte Kleeného větu.
- (b) **2 b** Sestrojte DFA pro jazyk slov nad $\{a, b\}$ obsahujících podслово ab . Uvedte uzávěrové vlastnosti regulárních jazyků (sjednocení, průnik, doplněk).
- (c) **3 b** Dokažte, že jazyk $L = \{a^n b^n \mid n \geq 0\}$ není regulární.

(a) $DFA = (Q, \Sigma, \delta, q_0, F)$ s $\delta : Q \times \Sigma \rightarrow Q$; přijímá w , skončí-li ve stavu z F . NFA má $\delta : Q \times \Sigma \rightarrow 2^Q$ (a případně ε -přechody); přijímá, existuje-li přijímající běh. *Regulární gramatika* = pravidla typu $A \rightarrow aB$ a $A \rightarrow a$ (resp. ε). *Regulární výraz* se tvoří z $\emptyset, \varepsilon$, symbolu operacemi $\cup, \cdot, *$. *Kleeného věta*: třídy jazyků popsatelných DFA, NFA, regulární gramatikou a regulárním výrazem jsou totožné (= regulární jazyky).

(b) DFA se 3 stavy, $F = \{q_2\}$:

	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

(q_0 = zatím nic, q_1 = naposledy a , q_2 = už viděno ab). *Uzávěry*: regulární jazyky jsou uzavřené na sjednocení a průnik (součinový automat na $Q_1 \times Q_2$), na doplněk (u DFA prohodíme $F \leftrightarrow Q \setminus F$), dále na zřetězení a iteraci.

(c) Sporem přes *pumping lemma*: necht L regulární s konstantou p . Vezmi $w = a^p b^p \in L$, $|w| \geq p$. Lemma dá rozklad $w = xyz$, $|xy| \leq p$, $|y| \geq 1$. Protože $|xy| \leq p$, leží y celé v úvodních a , tedy $y = a^k$, $k \geq 1$. Pak $xy^2z = a^{p+k}b^p \notin L$ (více a než b), což je spor. Tedy L není regulární.

Kalibrace: Automaty a jazyky jsou nejčastější okruh společné informatiky (78 % zkoušek, skoro vždy jako Q1). Konstrukce DFA + definice = spolehlivé 2 body; tato úloha je proto v jádru úrovně 1.

Pozor (častá chyba): Pumping lemma dokazuje neregularitu (sporem); neslouží k důkazu regularity. Vždy zdůvodni, proč y padne do bloku a (díky $|xy| \leq p$).

Úloha 10. **společná informatika** Bezkontextové jazyky, Turinguv stroj a nerozhodnutelnost

- 1 b** Definujte bezkontextovou gramatiku (CFG), jazyk jí generovaný a zásobníkový automat (PDA). Co je Chomského hierarchie (typy 0-3)?
- 2 b** Sestrojte CFG pro $\{a^n b^n \mid n \geq 0\}$ a popište princip převodu CFG na PDA.
- 3 b** Definujte Turinguv stroj a rekurzivně spočetné jazyky. Vysvětlete existenci algoritmicky nerozhodnutelného problému (problém zastavení).

(a) CFG $G = (N, \Sigma, P, S)$ s pravidly $A \rightarrow \alpha$, $A \in N$, $\alpha \in (N \cup \Sigma)^*$; *generovaný jazyk* = slova nad Σ odvoditelná z S . PDA = konečný automat se zásobníkem; třída jazyku přijímaných PDA = bezkontextové jazyky (CFL). *Chomského hierarchie*: typ 3 regulární $\subset$ typ 2 bezkontextové $\subset$ typ 1 kontextové $\subset$ typ 0 rekurzivně spočetné, s odpovídajícími stroji (KA, PDA, lineárně omezený automat, Turinguv stroj).

(b) CFG: $S \rightarrow aSb \mid \varepsilon$. *Převod CFG $\rightarrow$ PDA* (přijímání prázdným zásobníkem): na zásobník dej S ; v každém kroku buď nahraď nejvyšší neterminál pravou stranou pravidla (ε -přechod), nebo paruj nejvyšší terminál se vstupním symbolem (a oba odeber). Slovo je přijato, vyprázdní-li se zásobník při dočtení vstupu. PDA tak simuluje levou derivaci.

(c) *Turinguv stroj* = konečný řídicí automat s nekonečnou páskou, hlavou (čte/zapisuje/-posouvá), přechodovou funkcí; přijímá vstup, dostane-li se do přijímajícího stavu. Jazyk je *rekurzivně spočetný* (typ 0), přijímá-li ho nějaký TS (může cyklit na slovech mimo jazyk); *rekurzivní* (rozhodnutelný), pokud TS vždy zastaví. *Problém zastavení* (zastaví TS M na vstupu w ?) je nerozhodnutelný: kdyby existoval rozhodovač H , sestrojíme $D(x)$, který se zacyklí, právě když H řekne „ $x(x)$ zastaví“; pak $D(D)$ vede ke sporu (diagonalizace).

Kalibrace: Bezkontextové jazyky a Chomského hierarchie jsou jádro Q1; Turinguv stroj + nerozhodnutelnost jsou vzácnější, ale legální tah z téhož okruhu - tedy je držíme na definiční úrovni jako pojistku proti nule.

Pozor (častá chyba): „Bezkontextové“ = PDA, ne konečný automat. Nerozhodnutelnost se dokazuje diagonalizací/redukci, ne „je to těžké“.

Úloha 11. **společná informatika** Složitost, třídy P a NP, NP-úplnost

- (a) **1 b** Definujte časovou složitost a asymptotickou notaci (O, Ω, Θ). Definujte třídy P a NP.
- (b) **2 b** Definujte polynomiální převoditelnost, NP-těžkost a NP-úplnost. Uveďte aspoň tři NP-úplné problémy.
- (c) **3 b** Jak se dokazuje, že je problém NP-úplný? Co tvrdí Cookova-Levinova věta a co znamená otevřená otázka $P \stackrel{?}{=} NP$?

(a) *Časová složitost* = počet elementárních kroků jako funkce velikosti vstupu (nejhorší případ). $f = O(g)$: $\exists c, n_0 : f(n) \leq cg(n)$ pro $n \geq n_0$; Ω je dolní mez, $\Theta = O \cap \Omega$. P = problémy řešitelné deterministicky v polynomiálním čase. NP = problémy, jejichž „ano“ instance mají v polynomiálním čase ověřitelný certifikát (ekvivalentně řešitelné nedeterministicky v polynomiálním čase).

(b) *Polynomiální převod* $A \leq_p B$: funkce vyčíslitelná v polynomiálním čase, která instance A převádí na instance B se zachováním odpovědi. B je *NP-těžký*, pokud $A \leq_p B$ pro každé $A \in NP$; *NP-úplný*, je-li navíc $B \in NP$. Příklady NP-úplných: SAT, 3-SAT, klika, vrcholové pokrytí, barevnost grafu, Hamiltonovská kružnice, batoh (rozhodovací).

(c) NP-úplnost se dokazuje ve dvou krocích: (1) ukázat $B \in NP$ (certifikát + ověření); (2) zvolit známý NP-úplný problém A a sestavit polynomiální redukci $A \leq_p B$. *Cookova-Levinova věta*: SAT je NP-úplný (první NP-úplný problém), což dává startovní bod pro všechny další redukce. $P = NP$? je otevřená otázka, zda lze vše ověřitelné v polynomiálním čase i v polynomiálním čase *vyřešit*; obecně se věří, že $P \neq NP$.

Kalibrace: Algoritmy a složitost (P/NP) jsou opakovaný okruh; čistě definiční úloha s vysokou návratností bodu. Redukce z (b)/(c) přidá třetí bod.

Pozor (častá chyba): NP *není* „ne-polynomiální“. NP-těžkost nevyžaduje $B \in NP$; NP-úplnost ano. Směr redukce: ze *známého* těžkého na *nový* problém.

Úloha 12. společná informatika Rozděl a panuj, Master theorem, třídění

- (a) **1 b** Popište princip metody rozděl a panuj. Zformulujte Master theorem (kuchařkovou větu) a dolní odhad složitosti porovnávacího třídění.
- (b) **2 b** Popište Mergesort (pseudokód) a odvoďte jeho složitost z rekurentní rovnice. Uveďte princip a nejhorší/průměrný případ Quicksortu.
- (c) **3 b** Pomocí Master theorem určete asymptotiku pro $T(n) = 2T(n/2) + n$ a $T(n) = 3T(n/2) + n$. Co z toho plyne pro Karatsubovo násobení?

(a) *Rozděl a panuj*: rozděl úlohu na podproblémy, vyřeš rekurzivně, slož výsledky. *Master theorem*: pro $T(n) = aT(n/b) + f(n)$ ($a \geq 1, b > 1$) porovnej $f(n)$ s $n^{\log_b a}$:

- $f = O(n^{\log_b a - \varepsilon}) \Rightarrow T = \Theta(n^{\log_b a})$;
- $f = \Theta(n^{\log_b a}) \Rightarrow T = \Theta(n^{\log_b a} \log n)$;
- $f = \Omega(n^{\log_b a + \varepsilon})$ a regularita $\Rightarrow T = \Theta(f(n))$.

Dolní odhad porovnávacího třídění: $\Omega(n \log n)$ (rozhodovací strom má $n!$ listů, hloubka $\geq \log_2 n! = \Theta(n \log n)$).

(b) Mergesort: rozděl pole na poloviny, seříd rekurzivně, slij (MERGE) v $O(n)$. Rekurence $T(n) = 2T(n/2) + O(n)$, dle Master (případ 2) $T(n) = \Theta(n \log n)$; stabilní, $O(n)$ paměť navíc. Quicksort: zvol pivota, rozděl na menší/větší, rekurzivně; průměrně $O(n \log n)$, nejhůře $O(n^2)$ (špatný pivot), in-place.

(c) $T(n) = 2T(n/2) + n$: $a = b = 2$, $n^{\log_b a} = n$, $f = \Theta(n)$, případ 2 $\Rightarrow \Theta(n \log n)$. $T(n) =$

$3T(n/2) + n$: $\log_2 3 \approx 1,585$, $n^{\log_2 3}$ roste rychleji než $f = n$, případ $1 \Rightarrow \Theta(n^{\log_2 3})$. Karatsuba násobí n -ciferná čísla rekurzí $T(n) = 3T(n/2) + O(n)$, tedy $\Theta(n^{\log_2 3}) \approx n^{1,585}$ - lepší než školní $\Theta(n^2)$.

Kalibrace: Třídění a rozděl-a-panuj jsou opakovaný algoritmický okruh; Master theorem + Mergesort je věčný typový výpočet na 2-3 body.

Pozor (častá chyba): Master theorem porovnává $f(n)$ s $n^{\log_b a}$, ne s n . Quicksort má nejhorší případ $O(n^2)$ - nezaměňuj s průměrným.

Úloha 13. **společná informatika** Grafové algoritmy: prohledávání, nejkratší cesty, kostry, toky

- (a) **1 b** Popište prohledávání do šířky (BFS) a do hloubky (DFS), jejich složitost a reprezentaci grafu.
- (b) **2 b** Popište topologické třídění DAG a algoritmy pro nejkratší cesty: Dijkstra a Bellman-Ford (kdy který, složitost).
- (c) **3 b** Popište minimální kostru a algoritmy Jarník (Prim) a Borůvka (řezové lemma). Uveďte princip hledání maximálního toku (Ford-Fulkerson).

- (a) *BFS*: z počátku prohledává po vrstvách pomocí fronty; dává nejkratší cesty v neohodnoceném grafu. *DFS*: jde do hloubky pomocí zásobníku/rekurze; klasifikuje hrany (stromové, zpětné, dopředné, příčné), detekuje cykly. Obě $O(V + E)$ při reprezentaci *seznamy sousedu*.
- (b) *Topologické třídění* (jen DAG): lineární uspořádání vrcholu tak, že každá hrana vede „vpřed“; spočítá se opakovaným odebráním vrcholu se vstupním stupněm 0 nebo přes časy dokončení DFS, $O(V + E)$. *Dijkstra*: nejkratší cesty z jednoho zdroje pro *nezáporné* váhy, hladově s prioritní frontou, $O((V + E) \log V)$. *Bellman-Ford*: zvládá *záporné* hrany, $V - 1$ relaxací všech hran, $O(VE)$, navíc detekuje záporný cyklus.
- (c) *Minimální kostra* (MST) = kostra s minimálním součtem vah. *Jarník/Prim*: roste jeden strom, vždy přidá nejlevnější hranu ven, $O((V + E) \log V)$. *Borůvka*: každá komponenta paralelně přidá svou nejlevnější hranu, $O(E \log V)$. Korektnost plyne z *řezového lemmatu*: nejlevnější hrana přes libovolný řez patří do nějaké MST. *Ford-Fulkerson*: opakovaně hledej zlepšující cestu ze s do t v reziduální síti a posílej po ní tok, dokud existuje. Pro celočíselné kapacity vždy skončí; když už zlepšující cesta neexistuje, je tok maximální a jeho velikost se rovná kapacitě minimálního řezu.

Kalibrace: Grafové algoritmy jsou opakovaný informatický okruh (BFS/DFS, topo, MST, toky). Definice + jeden algoritmus s složitostí = 2 body; tady jde o breadth, ne hloubku důkazu.

Pozor (častá chyba): Dijkstra selže na záporných hranách - tam Bellman-Ford. Řezové lemma je klíč ke korektnosti MST; znej ho aspoň slovně.

Úloha 14. **společná informatika** Datové struktury: vyhledávací stromy a haldy

- (a) **1 b** Definujte binární vyhledávací strom (BVS) a jeho operace (find/insert/delete). Co je AVL strom a jakou má výšku?
- (b) **2 b** Popište binární (min-)haldy v poli a operace INSERT a EXTRACTMIN. Jak funguje Heapsort a jaká je jeho složitost?
- (c) **3 b** Popište hledání následníka (successor) v BVS. Porovnejte složitost operací u nevyváženého a vyváženého stromu. Proč jde halda postavit v $O(n)$?

(a) *BVS*: binární strom, kde pro každý uzel platí klíče vlevo < klíč uzlu < klíče vpravo. FIND/INSERT jdou shora dle porovnání; DELETE listu/uzlu s jedním synem je přímé, uzel se dvěma syny se nahradí svým následníkem. Vše $O(h)$ (výška). AVL je BVS udržující v každém uzlu rozdíl výšek podstromu ≤ 1 (rotacemi), takže $h = O(\log n)$.

(b) *Min-halda* = úplný binární strom v poli (uzel i má syny $2i+1, 2i+2$), kde rodič $\leq$ synové. INSERT: přidej na konec a „probublej nahoru” ($O(\log n)$). EXTRACTMIN: vrať kořen, dej poslední prvek na vrchol a „probublej dolů” ($O(\log n)$). Heapsort: postav haldu a n -krát vyber minimum/maximum; $O(n \log n)$, in-place.

(c) *Successor* (nejbližší větší): má-li uzel pravý podstrom, je to jeho nejlevnější uzel; jinak jdi nahoru, dokud nejsi levým synem rodiče (ten je následník). U *nevyváženého* BVS je h až $O(n)$ (degenerace na seznam), takže operace $O(n)$; u *vyváženého* (AVL) $O(\log n)$. Haldu lze postavit *zdola nahoru* (HEAPIFY od posledního vnitřního uzlu): součet prací $\sum_h \frac{n}{2^{h+1}} \cdot O(h) = O(n)$.

Kalibrace: Stromy a haldy jsou opakovaný okruh datových struktur (BVS, AVL, Heapsort se objevily v zadáních). Definice + operace s složitostí = 2 body.

Pozor (častá chyba): Mazání uzlu se dvěma syny v BVS jde přes následníka, ne ledabyly. Build-heap je $O(n)$, nikoli $O(n \log n)$ - klasická chytací otázka.

Úloha 15. **společná informatika** Reprezentace dat a čtení binárního souboru (C#)

Binární soubor má tento formát: 4 bajty ASCII „REC1” (magic), poté uint32 count (little-endian), poté count záznamů, kde každý záznam je int32 id (little-endian) následovaný float32 value (IEEE 754, little-endian).

- (a) **1b** Vysvětlete pojmy *little-endian* vs. *big-endian* a *zarovnání* (alignment). Jak je v paměti uloženo int32 s hodnotou 0x01020304 v little-endian?
- (b) **2b** Napište v C# metodu, která soubor načte do pole struktur (int Id, float Value).
- (c) **3b** Co se stane na big-endian stroji a jak to ošetřit? Jak ověřit magic a obránit se proti poškozenému/zkrácenému souboru?

(a) *Endianita* je pořadí bajtu vícebajtové hodnoty v paměti. *Little-endian* ukládá nejméně významný bajt na nejnižší adresu. Hodnota 0x01020304 se tedy v paměti jeví jako bajty 04 03 02 01 (od nízké adresy). Big-endian by uložil 01 02 03 04. *Zarovnání* znamená, že hodnota typu o velikosti s má typicky začínat na adrese dělitelné s (např. int32 na násobku 4); překladač proto vkládá výplň (padding) mezi položky struktury. V binárním *formátu souboru* se ale spoléháme na přesné pořadí bajtu a žádné implicitní zarovnání nepředpokládáme.

(b) BinaryReader v .NET čte v little-endian, což sedí na zadání:

```
public readonly record struct Rec(int Id, float Value);

public static Rec[] Load(string path)
{
    using var fs = File.OpenRead(path);
    using var br = new BinaryReader(fs);

    // magic "REC1"
    var magic = br.ReadBytes(4);
    if (magic.Length != 4 || magic[0] != (byte)'R' || magic[1] != (byte)'E'
        || magic[2] != (byte)'C' || magic[3] != (byte)'1')
        throw new InvalidDataException("spatny magic");

    uint count = br.ReadUInt32(); // little-endian
    var recs = new Rec[count];
```

```

for (uint i = 0; i < count; i++)
{
    int id = br.ReadInt32(); // 4 bajty LE
    float val = br.ReadSingle(); // 4 bajty IEEE754 LE
    recs[i] = new Rec(id, val);
}
return recs;
}

```

Ruční složení int32 z bajtu (ukázka principu): $\text{id} = \text{b0} \mid \text{b1} \ll 8 \mid \text{b2} \ll 16 \mid \text{b3} \ll 24$.

(c) Pozor na endianitu CPU: `BinaryReader` i explicitní složení $\text{b0} \mid \text{b1} \ll 8 \mid \text{b2} \ll 16 \mid \text{b3} \ll 24$ dávají *vždy* little-endian výsledek nezávisle na procesoru (to je správně a přenositelné). Co přenositelné *není*, je reinterpretace syrových bajtu v nativním pořadí CPU (např. `BitConverter.ToInt32` nebo přetypování ukazatele) - ta by na big-endian stroji vrátila prohozené bajty. Robustní je proto skládat bajty s pevným pořadím podle *formátu* (LE): v .NET `BinaryPrimitives.ReadInt32LittleEndian(span)` a `BinaryPrimitives.ReadSingleLittleEndian(span)`, případně podle `BitConverter.IsLittleEndian` bajty otočit. *Obrana proti poškození*: ověř magic; po načtení count zkontroluj, že zbývajících délka souboru je $= 8 \cdot \text{count}$ bajtu (jinak vyhoď výjimku) místo slepého čtení do EOF; `ReadBytes/ReadInt32` při zkrácení vyhodí `EndOfStreamException`, kterou je nutno ošetřit.

Kalibrace: Implementační otázky ze společné informatiky (binární soubor, alokátor, semafor, OO návrh) jsou hlavní příčina propadnutí „naoko připraveného“ studenta - na nule z této otázky se podlaha $\geq 5/12$ hroulí. Cíl: NIKDY nedostat 0. Část (a) je čistá definice (1 bod), kostra metody v (b) přidá druhý bod i bez doladění (c).

Pozor (častá chyba): `BinaryReader` čte vždy little-endian bez ohledu na CPU - na zkoušce to klidně řekni, ale dodej, že pro přenositelnost a big-endian CPU je správně `BinaryPrimitives`. Nezapomeň na ošetření konce souboru.

Úloha 16. společná informatika Architektura a OS: režimy, procesy, virtuální paměť

- (a) **1 b** Vysvětlíte adresu a adresový prostor, privilegovaný vs neprivilegovaný režim procesoru a roli jádra OS. Co je proces, vlákno, kontext a přepínání kontextu? Uveďte stavy vlákna.
- (b) **2 b** Vysvětlíte rozdíl virtuální vs fyzická adresa a komponenty stránkování (číslo stránky, číslo rámce, offset). Pro stránku 4 KiB přeložte virtuální adresu `0x00003ABC`, je-li stránka 3 namapována na rámec 7.
- (c) **3 b** Jakou roli hrají virtuální adresové prostory v ochraně paměti? Jak procesor realizuje konstrukce vyššího jazyka (přiřazení, podmínka, cyklus) instrukcemi?

(a) *Adresa* identifikuje místo v paměti; *adresový prostor* = množina platných adres (každý proces má vlastní). V *privilegovaném* (jádrovém) režimu smí procesor vše (I/O, správa paměti), v *neprivilegovaném* (uživatelském) jen omezeně; přechod přes přerušení/syscall. *Jádro OS* spravuje prostředky a izolaci. *Proces* = běžící program s vlastním adresovým prostorem; *vlákno* = tok provádění uvnitř procesu (sdílí paměť). *Kontext* = registry + čítač instrukcí + stav; *přepnutí kontextu* je uloží/obnoví. Stavy vlákna: *běží* (running), *připraveno* (ready), *čeká/blokováno* (waiting); plánovač přepíná preemptivně nebo kooperativně.

(b) *Virtuální* adresu používá program; *fyzická* ukazuje do RAM; překlad zajišťuje MMU přes stránkovací tabulku. $\text{Adresa} = (\text{číslo stránky} \parallel \text{offset})$. Stránka 4 KiB $\Rightarrow$ offset 12 bitu. `0x00003ABC`: offset = `0xABC`, číslo stránky = `0x00003ABC` $\gg 12 = 0x3 = 3$. Tabulka: stránka 3 $\rightarrow$ rámec 7. Fyzická adresa = $(7 \ll 12) \mid 0xABC = 0x7ABC$.

(c) Každý proces má vlastní mapování, takže nevidí cizí paměť (*ochrana/izolace*); přístup mimo

mapování vyvolá výpadek (page fault). Konstrukce vyššího jazyka: *přiřazení* = load/store mezi registrem a pamětí; *podmínka* = porovnání (cmp) + podmíněný skok; *cyklus* = podmíněný skok zpět; *volání funkce* = uložení návratové adresy a rámce na zásobník (call/ret).

Kalibrace: Architektura a OS jsou druhý nejčastější informatický okruh (60%); virtuální paměť a překlad adres se v zadáních opakují jako konkrétní výpočet. Tabulka adres je věčně 2-3 body.

Pozor (častá chyba): Offset má tolik bitů, kolik je $\log_2$ (velikost stránky); neplet číslo stránky (virtuální) s číslem rámce (fyzické). Stavby vlákna nejsou jen „běží/neběží”.

Úloha 17. **společná informatika** Synchronizace: kritická sekce, semaforey, producent-konzument (C#)

- (a) **1 b** Definujte časově závislou chybu (race condition), kritickou sekci a vzájemné vyloučení. Co je zámek a jaký je rozdíl mezi aktivním a pasivním čekáním?
- (b) **2 b** Co je semafor (operace WAIT/P a SIGNAL/V)? Napište v C# řešení producent-konzument s omezeným bufferem pomocí semaforu.
- (c) **3 b** Uvedte čtyři podmínky uvážnutí (deadlock). Porovnejte spinlock a mutex a vysvětlete atomickou operaci compare-and-swap (CAS).

(a) *Race condition*: výsledek závisí na nedeterministickém prokládání vláken nad sdíleným stavem. *Kritická sekce*: úsek kódu, který smí běžet nejvýše jedno vlákno najednou. *Vzájemné vyloučení* (mutual exclusion) tuto vlastnost zajišťuje. *Zámek* (lock) chrání kritickou sekci; *aktivní čekání* (spin) cyklí a pálí CPU, *pasivní* vlákno uspí a plánovač ho probudí (lepší při delším čekání).

(b) *Semafor* = čítač s atomickými operacemi: WAIT/P sníží čítač (a zablokuje při 0), SIGNAL/V zvýší (a probudí čekající). Omezený buffer kapacity N :

```
// empty: kolik je volných míst, full: kolik je obsazených
var empty = new SemaphoreSlim(N, N);
var full = new SemaphoreSlim(0, N);
var mutex = new object();
var buffer = new Queue<int>();

void Producer(int item) {
    empty.Wait(); // počkej na volné místo (P)
    lock (mutex) { buffer.Enqueue(item); }
    full.Release(); // přibyl prvek (V)
}

int Consumer() {
    full.Wait(); // počkej na prvek (P)
    int item;
    lock (mutex) { item = buffer.Dequeue(); }
    empty.Release(); // uvolnil se slot (V)
    return item;
}
```

(c) *Deadlock* nastane při současném splnění: (1) vzájemné vyloučení, (2) drž a čekej, (3) bez odebrání (no preemption), (4) kruhové čekání. *Spinlock* aktivně čeká (vhodný pro velmi krátké sekce na více jádrech), *mutex* uspí vlákno (vhodný pro delší). *CAS* (CompareAndSwap(addr, expected, new)): atomicky porovná a případně zapíše; základ neblokujících (lock-free) struktur.

Kalibrace: Synchronizace (semaforey, kritická sekce, race condition) je nejčastější OS podtéma a často chce kód. Definice (a) = jistý bod, kostra semaforu (b) = druhý; tím se vyhneš nule na implementační otázce.

Pozor (častá chyba): Pořadí semaforu u producent-konzument je klíčové: čekej na zdroj (WAIT) PŘED zámkem, jinak hrozí deadlock. empty a full se nesmí prohodit.

Úloha 18. společná informatika Objektové programování a návrh (C#)

- (a) **1b** Vysvětlete abstrakci, zapouzdření a polymorfismus a související konstrukty (třída, rozhraní, dědičnost, viditelnost). Rozdíl mezi statickým a dynamickým typováním a mezi virtuální a nevirtuální metodou.
- (b) **2b** Navrhněte v C# malou hierarchii geometrických tvarů s rozhraním a polymorfním výpočtem obsahu; ukažte ošetření chyby výjimkou.
- (c) **3b** Jak je implementován dynamický polymorfismus (tabulka virtuálních metod)? K čemu jsou generické typy a jaký je rozdíl mezi hodnotovými a referenčními typy v C#?

(a) *Abstrakce* skrývá detaily za rozhraním; *zapouzdření* drží data + operace pohromadě a omezuje přístup (viditelnost `private/public/protected`); *polymorfismus* = jednotné volání nad různými typy. *Třída* definuje typ, *rozhraní* (interface) jen kontrakt, *dědičnost* přebírá a rozšiřuje. *Statické* typování kontroluje typy při překladu, *dynamické* za běhu. *Virtuální* metoda se vybírá podle skutečného typu objektu za běhu (override), *nevirtuální* podle deklarovaného typu.

(b)

```
public interface IShape { double Area(); }

public sealed class Circle : IShape {
    private readonly double r;
    public Circle(double r) {
        if (r < 0) throw new ArgumentException("zaporný polomer");
        this.r = r;
    }
    public double Area() => Math.PI * r * r;
}

public sealed class Rectangle : IShape {
    private readonly double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double Area() => w * h;
}

double TotalArea(IEnumerable<IShape> shapes) {
    double sum = 0;
    foreach (var s in shapes) sum += s.Area(); // polymorfni volani
    return sum;
}
```

(c) *Dynamický polymorfismus* se realizuje *tabulkou virtuálních metod* (vtable): každý objekt drží ukazatel na tabulku své třídy, volání virtuální metody = skok přes položku tabulky (vybere se override potomka). *Generické typy* umožní psát kód nezávislý na konkrétním typu (`List<T>`) bez ztráty typové kontroly a bez boxingu u hodnotových typu. *Hodnotové typy* (`struct`, `int`) se kopírují podle hodnoty (typicky na zásobníku), *referenční* (`class`) žijí na haldě a předává se reference; spravuje je garbage collector.

Kalibrace: Programovací jazyky / OO návrh je v moderním formátu vzácnější, ale když padne, bývá implementační. Definice (a) + funkční kostra hierarchie (b) tě udrží nad nulou; jazyk je C# (volba majitele).

Pozor (častá chyba): Rozhraní není abstraktní třída (nemá stav/implementaci, lze jich implementovat víc). Virtuální dispatch jde podle *skutečného* typu - jinak by polymorfismus nefungoval.

3 Specializace: Strojové učení (Téma 3)

Úloha 19. specializace UI-SU Lineární a logistická regrese, regularizace

- (a) **1 b** Definujte model lineární regrese a chybovou funkci (MSE). Napište *normální rovnice* a analytické řešení metodou nejmenších čtverců.
- (b) **2 b** Co je L2 (ridge) regularizace? Jak změní normální rovnice a jaký má vliv na koeficienty? Proč pomáhá proti přeučení a kdy je nutná (singularita $X^T X$)?
- (c) **3 b** Logistická regrese pro binární klasifikaci: napište predikci (sigmoid), ztrátovou funkci (NLL / křížová entropie) a gradient pro jeden krok SGD. Nemusíte nic počítat numericky.

(a) **Lineární regrese.** Data $X \in \mathbb{R}^{n \times d}$ (řádky x_i), cíle $y \in \mathbb{R}^n$, model $\hat{y} = Xw$ (sloupec jedniček v X dává bias). Chyba:

$$\text{MSE}(w) = \frac{1}{n} \sum_{i=1}^n (x_i^T w - y_i)^2 = \frac{1}{n} \|Xw - y\|^2.$$

Minimum: $\nabla_w = \frac{2}{n} X^T (Xw - y) = 0$, tj. *normální rovnice*

$$X^T X w = X^T y \Rightarrow w = (X^T X)^{-1} X^T y \text{ (pokud je } X^T X \text{ regulární)}.$$

(b) **L2 / ridge.** Minimalizujeme $\|Xw - y\|^2 + \lambda \|w\|^2$, $\lambda > 0$. Normální rovnice se změní na

$$(X^T X + \lambda I) w = X^T y, \quad w = (X^T X + \lambda I)^{-1} X^T y.$$

Matice $X^T X + \lambda I$ je vždy regulární (pozitivně definitní), takže řešení existuje i při kolineárních příznacích nebo $d > n$, kdy je $X^T X$ singularní. Regularizace *smršťuje* koeficienty směrem k nule (snižuje rozptyl modelu za cenu malého vychýlení), čímž omezuje přeučení. Větší λ = silnější smrštění; λ se volí křížovou validací.

(c) **Logistická regrese.** Predikce pravděpodobnosti třídy 1:

$$p_i = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}.$$

Ztráta (negativní log-věrohodnost / binární křížová entropie):

$$L(w) = - \sum_{i=1}^n [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$

Gradient má jednoduchý tvar $\nabla_w L = \sum_i (p_i - y_i) x_i$. Krok SGD na jednom příkladu (x_i, y_i) :

$$w \leftarrow w - \eta (p_i - y_i) x_i,$$

kde η je rychlost učení. (Pro více tříd: softmax místo sigmoidu, ztráta = kategoriální křížová entropie, gradient $(p_i - y_i) x_i$.)

Lineární: normální rovnice $X^T X w = X^T y$. Ridge: $(X^T X + \lambda I) w = X^T y$ (smršťuje, vždy řešitelné). Logistická: $p = \sigma(w^T x)$, ztráta NLL, gradient $\sum_i (p_i - y_i) x_i$, SGD krok $w \leftarrow w - \eta (p_i - y_i) x_i$.

Kalibrace: Regrese je nejčastější ML okruh (67% zkoušek) a sytí kritickou podlahu specializace ($\geq 7/12$, kde

potřebuješ tři ze čtyř otázek na ≥ 2 body). Sigmoid, NLL a normální rovnice umět z paměti = spolehlivé 2-3 body.

Pozor (častá chyba): „Metoda nejmenších čtverců“ = analytické řešení přes normální rovnice; SGD je iterativní alternativa. Pleteš-li si je, ztrácíš bod. U ridge nezapomeň, že bias (w_0) se obvykle neregularizuje.

Úloha 20. specializace UI-SU Shlukování: k-means a hierarchické shlukování

- (a) **1 b** Definujte úlohu shlukování (učení bez učitele) a algoritmus k-means (krok přiřazení a krok aktualizace) a jeho účelovou funkci.
- (b) **2 b** Co je inicializace k-means++? Proč ztráta v k-means monotónně klesá a proč algoritmus konverguje (do lokálního minima)? Popište aglomerativní hierarchické shlukování a dendrogram.
- (c) **3 b** Jak zvolit počet shluku k ? Proč je k-means citlivý na škálování příznaku? Proveďte jeden krok k-means pro body 1, 2, 10, 11 na přímce s centry $c_1 = 0, c_2 = 20$.

- (a) *Shlukování*: rozděl neoznačená data do k skupin podle podobnosti. k -means minimalizuje vnitroshlukový součet čtvercu $J = \sum_i \|x_i - \mu_{c(i)}\|^2$ střídáním: (1) *přiřazení* každého bodu k nejbližšímu centru; (2) *aktualizace* center na průměr přiřazených bodů.
- (b) k -means++: počáteční centra volí postupně s pravděpodobností úměrnou $D(x)^2$ (vzdálenost k nejbližšímu už zvolenému centru), což snižuje riziko špatného startu. Oba kroky J nezvyšují (přiřazení k nejbližšímu i průměr jsou optimální dílčí kroky), J je zdola omezené, konfigurací je konečně mnoho $\Rightarrow$ konvergence, ale jen do *lokálního* minima (proto víc restartů). *Aglomerativní* shlukování začíná s každým bodem zvlášť a opakovaně spojuje dva nejbližší shluky (linkage: single/complete/average); historii zachytí *dendrogram*, který se „uřízne“ na požadovaný počet shluku.
- (c) k se volí např. metodou lokte (zlom v poklesu J) nebo silhouette skórem. *Škálování*: euklidovská vzdálenost míchá jednotky, takže příznak s velkým rozsahem dominuje; proto se data standardizují. Krok pro body $\{1, 2, 10, 11\}$, $c_1 = 0, c_2 = 20$: přiřazení: 1, 2, 10, 11 jsou blíže c_1 než c_2 ? $|10 - 0| = |10 - 20| = 10$ (remíza), $|11 - 0| = 11 > |11 - 20| = 9$, takže 11 jde k c_2 . Tedy $\{1, 2, 10\} \rightarrow c_1, \{11\} \rightarrow c_2$ (remízu u 10 dáme c_1). Aktualizace: $c_1 = \frac{1+2+10}{3} = 4,33, c_2 = 11$. (Další iterace už rozdělí na $\{1, 2\}$ a $\{10, 11\}$.)

k-means minimalizuje $J = \sum_i \|x_i - \mu_{c(i)}\|^2$ střídáním: (1) přiřaď bod k nejbližšímu centru, (2) přepočti centra na průměr. Konverguje jen do lokálního minima (víc restartů, k-means++), škáluj data. Hierarchické = spojuj nejbližší shluky, dendrogram.

Kalibrace: Shlukování je druhý nejčastější ML okruh (44 %, k-means se opakuje). Definice + jeden krok výpočtu = 2 body do kritické podlahy specializace.

Pozor (častá chyba): k-means najde jen *lokální* minimum a předpokládá zhruba kulové shluky; není deterministický (závisí na inicializaci). Před během škáluj příznaky.

Úloha 21. specializace UI-SU Evaluační metriky binárního klasifikátoru

- (a) **1 b** Nakreslete matici záměn a definujte accuracy, precision, recall a F1.
- (b) **2 b** Proč je accuracy zavádějící u nevyvážených tříd? Kdy upřednostnit precision a kdy recall? Z hodnot TPR, FPR a počtu pozitivních/negativních rekonstruuje TP, FP, TN, FN.
- (c) **3 b** Co zobrazuje ROC křivka a co je AUC? Spočítejte precision a recall pro TP = 20, FP = 30, FN = 5.

(a) Matice záměn (řádky skutečnost, sloupce predikce):

	$\hat{+}$	$\hat{-}$
$+$	TP	FN
$-$	FP	TN

accuracy = $\frac{TP+TN}{\text{vše}}$, precision = $\frac{TP}{TP+FP}$, recall (TPR) = $\frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$ (harmonický průměr).

(b) Při nevyvážených třídách (např. 1 % pozitivních) dá triviální klasifikátor „vše negativní“ accuracy 99 %, ač nic nenajde - proto se sleduje precision/recall. *Recall* je důležitý, když je drahé minout pozitivní (nemoc, podvod); *precision*, když je drahý falešný poplach (spam označený jako důležitý). Rekonstrukce z TPR, FPR a počtu P (pozitivních) a N (negativních): $TP = TPR \cdot P$, $FN = P - TP$, $FP = FPR \cdot N$, $TN = N - FP$.

(c) *ROC* vynáší TPR proti FPR při měnícím se prahu klasifikace; *AUC* (plocha pod ní) je pravděpodobnost, že náhodný pozitivní dostane vyšší skóre než náhodný negativní (1 = ideál, 0,5 = náhoda). Pro zadání: precision = $\frac{20}{20+30} = 0,4$, recall = $\frac{20}{20+5} = 0,8$.

precision = $\frac{TP}{TP+FP}$, recall = $\frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$ (harmonický). U nevyvážených tříd accuracy klame. Rekonstrukce: $TP = TPR \cdot P$, $FP = FPR \cdot N$. ROC = TPR vs FPR.

Kalibrace: Evaluační metriky jsou velmi častý ML okruh (44 %) a opakovaně chtějí rekonstrukci počtu z metrik. Vzorce $P/R/F_1$ a převod TPR/FPR umět z paměti = spolehlivé body.

Pozor (častá chyba): Precision a recall si neplet (jmenovatel: u precision predikované pozitivní, u recall skutečně pozitivní). F_1 je harmonický, ne aritmetický průměr.

Úloha 22. specializace UI-SU Rozhodovací stromy: kritéria větvení a prořezávání

- (a) **1 b** Definujte rozhodovací strom a kritéria větvení: entropii a informační zisk, Gini index.
- (b) **2 b** Popište hladovou konstrukci stromu. Spočítejte informační zisk štěpení, které z uzlu se 4 kladnými a 4 zápornými příklady udělá dva čisté listy (4/0 a 0/4).
- (c) **3 b** Co je přeučení u stromu a jak mu brání prořezávání (pre-pruning vs post-pruning)?

(a) *Rozhodovací strom*: vnitřní uzly testují příznak, listy přiřazují třídu (či hodnotu). *Entropie* množiny S s podíly tříd p_k : $H(S) = -\sum_k p_k \log_2 p_k$. *Informační zisk* štěpení S na části S_j : $IG = H(S) - \sum_j \frac{|S_j|}{|S|} H(S_j)$. *Gini*: $G(S) = 1 - \sum_k p_k^2$ (alternativní míra nečistoty).

(b) *Konstrukce (ID3/CART)*: v každém uzlu zvol příznak s největším informačním ziskem (resp. nejmenší váženou nečistotou), rozděl data a rekurzivně pokračuj, dokud nejsou listy čisté nebo nenastane zastavení. Výpočet: rodič 4+/4- má $H = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$ bit. Děti 4/0 a 0/4 jsou čisté: $H = 0$. $IG = 1 - (\frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 0) = 1$ bit (maximální možný zisk - dokonalé rozdělení).

(c) *Přeučení*: strom roste do hloubky, učí se šum a na trénovacích datech má nulovou chybu, ale špatně generalizuje. *Pre-pruning* (předčasné zastavení): omezení hloubky, minimální počet vzorku v listu, minimální zisk. *Post-pruning*: nech strom dorůst a pak ořezávej podstromy, které nezlepšují chybu na validační množině (nahrazení listem).

Entropie $H(S) = -\sum_k p_k \log_2 p_k$, informační zisk $IG = H(S) - \sum_j \frac{|S_j|}{|S|} H(S_j)$ (vážený), Gini $1 - \sum p_k^2$. Greedy konstrukce podle max IG, prořezávání proti přeučení.

Kalibrace: Rozhodovací stromy jsou častý okruh (33%) a vděčí za body za entropii/Gini a jeden výpočet IG. Patří do plně zvládnutého jádra specializace.

Pozor (častá chyba): Entropie se počítá s $\log_2$ (bity). Informační zisk je *vážený* podle velikostí dětí, ne prostý rozdíl entropií.

Úloha 23. specializace UI-SU Kombinace modelu: bagging, boosting, náhodné lesy

- 1 b** Co je kombinace více modelu (ensemble)? Vysvětlete rozdíl mezi baggingem a boostingem.
- 2 b** Popište náhodný les (random forest) a vysvětlete, proč zlepšuje přesnost oproti jednomu stromu.
- 3 b** Popište princip AdaBoostu. Za jakých podmínek ensemble pomáhá nejvíc a kdy vůbec ne (majoritní hlasování)?

- Ensemble* kombinuje predikce více modelu (hlasování/průměr) do silnějšího. *Bagging* trénuje modely *nezávisle* na bootstrap vzorcích a průměruje (snižuje hlavně *rozptyl*). *Boosting* trénuje modely *postupně*, každý se zaměří na chyby předchozích (snižuje hlavně *vychýlení*).
- Random forest* = bagging rozhodovacích stromu, kde se navíc v každém štěpení uvažuje jen náhodná podmnožina příznaku. To stromy *dekoreluje*; protože průměr nekorelovaných chyb má menší rozptyl, les generalizuje lépe než jeden hluboký (přeucený) strom, a přitom téměř nepotřebuje ladění.
- AdaBoost*: udržuje váhy příkladu, v každém kole natrénuje slabý klasifikátor, zvýší váhy špatně klasifikovaných a přidá klasifikátor s váhou podle jeho přesnosti; finální predikce = vážené hlasování. Ensemble pomáhá nejvíc, jsou-li dílčí modely *lepší než náhoda* a dělají *různé/nekorelované* chyby. Naopak nepomůže, dělají-li všechny stejné chyby (dokonale korelované) - pak majoritní hlasování dopadne stejně jako jeden model; nejlepší případ je, když se chyby nepřekrývají a většina vždy „přehlasuje“ menšinu chybujících.

Bagging = trénuj nezávisle na bootstrap vzorcích + průměruj (snižuje rozptyl). Boosting = sekvenčně, každý model na chybách předchozích (snižuje vychýlení). Random forest = bagging stromu + náhodná podmnožina příznaku (dekorelace). Pomáhá při nekorelovaných chybách.

Kalibrace: Kombinace modelu (22%) je vděčná konceptuální otázka (žádný výpočet). *Bagging vs boosting + random forest* = 2-3 body za vysvětlení.

Pozor (častá chyba): Bagging $\neq$ boosting: bagging paralelně/nezávisle (rozptyl), boosting sekvenčně na chybách (vychýlení). Les pomáhá díky *dekorelaci* stromu, ne jen jejich počtu.

Úloha 24. specializace UI-SU Metoda podpůrných vektorů (SVM)

- 1 b** Popište SVM pro lineárně separabilní třídy: co je rozdělovací nadrovina, margin a podpůrné vektory (support vectors)?
- 2 b** Co je měkký margin a parametr C ? Jaký je kompromis mezi šířkou marginu a počtem chyb?
- 3 b** K čemu slouží jádrové funkce (kernel trick) a jak se chová RBF jádro? Jak SVM

klasifikuje do více tříd?

- (a) SVM hledá *rozdělující nadrovinu* $w^\top x + b = 0$ s *maximálním marginem* (pásem) mezi třídami. Margin je vzdálenost nadroviny k nejbližším bodům; ty body leží na okraji a nazývají se *podpůrné vektory* - jen ony určují řešení (ostatní lze odebrat beze změny). Maximalizace marginu = minimalizace $\frac{1}{2}\|w\|^2$ za podmínek $y_i(w^\top x_i + b) \geq 1$.
- (b) Když data nejsou separabilní, zavede se *měkký margin* s prom. uvolnění $\xi_i \geq 0$: minimalizujeme $\frac{1}{2}\|w\|^2 + C \sum_i \xi_i$. Parametr C váží chyby proti šířce marginu: *velké* C = málo chyb, úzký margin (riziko přeučení); *malé* C = širší margin, toleruje víc chyb (lepší generalizace).
- (c) *Jádrový trik*: nahradí skalární součin $x_i^\top x_j$ jádrem $K(x_i, x_j)$, čímž SVM počítá v (implicitně vysokorozměrném) prostoru rysu, aniž ho explicitně sestrojí - umožní nelineární hranice. *RBF* $K(x, z) = \exp(-\gamma\|x - z\|^2)$ měří podobnost lokálně; malé γ = hladká hranice, velké γ = klikatá (přeučení). *Více tříd*: kombinací binárních klasifikátorů one-vs-rest nebo one-vs-one.

SVM maximalizuje margin: $\min \frac{1}{2}\|w\|^2$ za $y_i(w^\top x_i + b) \geq 1$; hranici určují podpůrné vektory. Měkký margin: $+C \sum \xi_i$ (C váží chyby vs šířku marginu). Jádro (RBF) = nelineární hranice bez explicitního prostoru rysu.

Kalibrace: SVM (12%) je vzácnější, ale když padne, je konceptuální („popište, nakreslete hranici, vliv bodu, kdy RBF“). Definice marginu + role C /jádra = 2-bodová záloha.

Pozor (častá chyba): Hranici určují jen podpůrné vektory. Velké C = méně tolerance chyb (užší margin), ne naopak; RBF s velkým γ se přeučuje.

Úloha 25. specializace UI-SU Analýza hlavních komponent (PCA)

- (a) **1b** Co je PCA a k čemu slouží? Co jsou hlavní komponenty a jak souvisí s kovarianční maticí?
- (b) **2b** Popište postup PCA (centrování, kovariance, vlastní rozklad / SVD, projekce na top- k). Co je vysvětlený rozptyl?
- (c) **3b** Jaká je souvislost PCA a SVD? Proč je nutné data před PCA škálovat a co PCA optimalizuje (rekonstrukční chyba)?

- (a) PCA je lineární redukce dimenze: hledá nové ortogonální osy (*hlavní komponenty*), podél nichž mají data největší rozptyl, a projektuje na prvních k z nich. Hlavní komponenty jsou *vlastní vektory kovarianční matice* dat; příslušná vlastní čísla udávají rozptyl podél nich.
- (b) Postup: (1) *centruj* data (odečti průměr), případně standardizuj; (2) spočti *kovarianční matici* $C = \frac{1}{n}X^\top X$; (3) najdi její *vlastní čísla a vektory* (nebo SVD matice X); (4) seřaď komponenty podle vlastních čísel a *projektuj* na prvních k . *Vysvětlený rozptyl* komponenty = $\lambda_i / \sum_j \lambda_j$; volíme k tak, aby pokryl např. 95% rozptylu.
- (c) Pro centrovaná data $X = U\Sigma V^\top$ (SVD) jsou hlavní komponenty sloupce V a singulární hodnoty σ_i souvisí s vlastními čísly kovariance přes $\lambda_i = \sigma_i^2/n$; SVD je numericky stabilnější cesta. *Škálování* je nutné, protože PCA reaguje na rozptyl: příznak s velkou jednotkou (rozsahem) by jinak uměle dominoval první komponentě. PCA minimalizuje *rekonstrukční chybu* (součet čtverců kolmých vzdáleností k podprostoru), což je ekvivalentní maximalizaci zachyceného rozptylu.

Hlavní komponenty = vlastní vektory kovarianční matice (seřazené podle vlastních čísel = rozptylu), projekce na top- k . Postup: centruj/škáluj, kovariance, vlastní rozklad / SVD, projekce. Maximalizuje rozptyl = minimalizuje rekonstrukční chybu.

Kalibrace: PCA (12 %) je vzácnější, ale opakované jako konceptuální + vizuální otázka. Definice (komponenty = vlastní vektory kovariance) + postup = 2-bodová záloha.

Pozor (častá chyba): PCA hledá směry maximálního rozptylu, ne směry oddělující třídy (to je LDA). Bez centrování/škálování vyjdou komponenty zkreslené.

Úloha 26. specializace UI-SU k-nejbližších sousedu (kNN)

- (a) **1 b** Popište algoritmus k-nejbližších sousedu pro klasifikaci i regresi. Proč se mu říká „líné“ učení (lazy)?
- (b) **2 b** Jak se volí k (křížová validace) a jak malé vs velké k ovlivní výsledek (vychýlení vs rozptyl)? Proč je nutné škálovat příznaky?
- (c) **3 b** Co je prokletí dimenzionality a jak postihuje kNN? Jaká je výpočetní složitost predikce?

(a) kNN si *pamatuje* trénovací data; pro nový bod najde k nejbližších (dle metriky, typicky euklidovské) a *klasifikace* = většinové hlasování jejich tříd, *regrese* = průměr jejich hodnot. Je *líné*, protože netrénuje žádný model - veškerá práce je až při predikci.

(b) k se volí *křížovou validací*. *Malé k* (např. 1) dává nízké vychýlení, ale vysoký rozptyl (citlivé na šum, přeučení); *velké k* hranici vyhladí (vyšší vychýlení, nižší rozptyl). *Škálování* je nutné, protože vzdálenost jinak ovládne příznak s největším rozsahem.

(c) *Prokletí dimenzionality*: ve vysoké dimenzi se vzdálenosti mezi body vyrovnávají (poměr nejbližší/nejvzdálenější $\rightarrow 1$), takže pojem „soused“ ztrácí smysl a kNN selhává; navíc je třeba exponenciálně víc dat. *Složitost* naivní predikce je $O(nd)$ na jeden dotaz (projít n bodu v dimenzi d); urychlení přes k -d stromy či přibližné hledání.

kNN = ulož data, pro nový bod najdi k nejbližších; klasifikace = hlasování, regrese = průměr. Líné učení. k přes křížovou validaci (malé k = rozptyl, velké = vychýlení). Škáluj příznaky; trpí prokletím dimenzionality.

Kalibrace: kNN (12 %) je vzácnější, ale jednoduchý a vděčný. Definice + role k a škálování = spolehlivá 2-bodová záloha; navíc nese pojem prokletí dimenzionality.

Pozor (častá chyba): kNN netrénuje (lazy) - „trénink“ je jen uložení dat. Bez škálování dominuje příznak s velkou jednotkou; pozor i na liché k u dvou tříd (remízy).

Úloha 27. specializace UI-SU Statistické testy: t-test a chí-kvadrát

- (a) **1 b** Co je statistický test, nulová hypotéza, hladina významnosti a p-hodnota? Popište jednovýběrový a dvouvýběrový t-test.
- (b) **2 b** Popište chí-kvadrát test dobré shody: testovou statistiku, stupně volnosti a rozhodovací pravidlo.
- (c) **3 b** Co je párový t-test a kdy ho použít? Jak souvisí rozhodnutí testu s chybami 1. a 2. druhu?

- (a) Test rozhoduje mezi nulovou hypotézou H_0 a alternativou na základě dat. Hladina významnosti α = přípustná pravděpodobnost chyby 1. druhu; p -hodnota = pravděpodobnost dat aspoň tak extrémních jako pozorovaná, platí-li H_0 ; zamítáme, je-li $p < \alpha$. Jednovýběrový t-test: $H_0 : \mu = \mu_0$, statistika $t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$, která má za platnosti H_0 Studentovo t -rozdělení s $n - 1$ stupni volnosti; zamítáme, padne-li t do kritického oboru (resp. $p < \alpha$). Dvouvýběrový: $H_0 : \mu_1 = \mu_2$, statistika $t = \frac{\bar{x} - \bar{y}}{\sqrt{s_X^2/n + s_Y^2/m}}$ (Welchova varianta při různých rozptylech).
- (b) Chí-kvadrát test dobré shody porovná pozorované četnosti O_i s očekávanými E_i podle modelu: $\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$. Stupně volnosti = (počet kategorií - 1 - počet odhadovaných parametru). H_0 (data odpovídají modelu) zamítneme, je-li χ^2 větší než kritická hodnota $\chi^2_{1-\alpha}$ pro dané df.
- (c) Párový t-test se použije, jsou-li data spárovaná (např. měření před/po na týchž jedincích); testuje, zda je průměr rozdílu nulový. Chyba 1. druhu = zamítnout platné H_0 (řízena α); chyba 2. druhu = nezamítnout neplatné H_0 (řízena silou testu $1 - \beta$, roste s velikostí vzorku a efektu).

Test: H_0 , testová statistika, p -hodnota; zamítni při $p < \alpha$. Jednovýběrový t-test $t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$. Chí-kvadrát dobré shody $\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$, df = kategorie - 1 - parametry.

Kalibrace: Statistické testy (22 % dohromady) chtějí převážně definice + dosazení do statistiky. Stačí 2-bodová záloha: hypotéza, statistika, rozhodnutí.

Pozor (častá chyba): p -hodnota není pravděpodobnost, že H_0 platí. Stupně volnosti u chí-kvadrát nejsou počet kategorií, ale po odečtení vazeb/parametru.

Úloha 28. specializace UI-SU Vyhodnocení modelu: validace, přeučení, regularizace

- (a) **1 b** Vysvětlete rozdělení dat na train/dev/test a křížovou validaci (k -fold). Co je princip maximální věrohodnosti?
- (b) **2 b** Co je přeučení (overfitting) a generalizační chyba? Jak proti přeučení působí regularizace (L1 vs L2) a včasné zastavení trénování?
- (c) **3 b** Vysvětlete kompromis vychýlení-rozptyl (bias-variance) a prokletí dimenzionality při výběru modelu.

- (a) *Train* slouží k učení, *dev/validace* k ladění hyperparametru a výběru modelu, *test* k jedinému závěrečnému odhadu výkonu (nesmí se na něm ladit). k -fold křížová validace: data rozděl na k částí, k -krát trénuj na $k-1$ a validuj na zbylé, výsledky zprůměruj (lepší využití dat). Maximální věrohodnost: zvol parametry maximalizující pravděpodobnost (věrohodnost) pozorovaných dat, $\hat{\theta} = \arg \max_{\theta} P(\text{data} | \theta)$.
- (b) *Přeučení*: model se naučí šum trénovacích dat, má malou trénovací, ale velkou generalizační (testovací) chybu. Regularizace přidá penaltu za velikost vah: $L2$ ($\lambda \|w\|_2^2$) hladce smršťuje, $L1$ ($\lambda \|w\|_1$) tlačí váhy přesně na nulu (řídke řešení, výběr příznaku). Včasné zastavení: trénink se ukončí, když začne růst validační chyba.
- (c) *Bias-variance*: příliš jednoduchý model má velké vychýlení (podučení), příliš složitý velký rozptyl (preučení); cílem je minimum jejich součtu. Prokletí dimenzionality: s rostoucím počtem příznaku roste potřeba dat exponenciálně a vzdálenosti se vyrovnávají, takže složitější modely snáz přeučí - proto redukce dimenze/regularizace.

Train (učení) / dev (ladění) / test (jediný závěrečný odhad); k -fold CV průměruje. Přeučení = malá train, velká test chyba. L2 hladce smršťuje, L1 dělá řídké váhy (výběr příznaku). Bias-variance: minimalizuj součet.

Kalibrace: Toto je „zastřešující“ ML téma (evaluace, regularizace) prolínající mnoho otázek; opakovaně se ptá na křížovou validaci a přeučení. Solidní definice = body napříč specializací.

Pozor (častá chyba): Na testovacích datech se *neladí*. L1 dělá řídké váhy (výběr příznaku), L2 jen smršťuje - nezaměňuj jejich efekt.

Úloha 29. specializace UI-SU Predikce z tabulárních dat: předzpracování a metriky

- (a) **1 b** Popište předzpracování tabulárních dat: kódování kategoriálních příznaku (one-hot), škálování spojitých příznaku a ošetření chybějících hodnot (imputace).
- (b) **2 b** Definujte regresní metriky MAE, RMSE a R^2 . Kdy je vhodnější MAE a kdy RMSE?
- (c) **3 b** Jaký model zvolit pro tabulární regresi (lineární / stromové / náhodný les / boosting) a proč stromové metody nepotřebují škálování?

(a) *Kategoriální* příznaky se převedou na čísla: *one-hot* kódování (každá hodnota = binární sloupec) pro neuspořádané kategorie; *ordinální* kódování pro uspořádané. *Spojité* příznaky se *škálují* (standardizace na nulový průměr a jednotkový rozptyl, nebo min-max) - nutné pro vzdálenostní a gradientové modely. *Chybějící hodnoty*: imputace (průměr/medián/nejčastější hodnota), případně příznak „bylo chybějící“.

(b) Pro predikce $\hat{y}_i$ a cíle y_i : $MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$; $RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$; $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ (podíl vysvětleného rozptylu). *MAE* je odolnější vůči odlehlým hodnotám; *RMSE* velké chyby trestá kvadraticky (preferuj, když jsou velké chyby zvlášť nežádoucí).

(c) *Lineární regrese* pro zhruba lineární vztahy (rychlá, interpretovatelná); *stromové metody* (rozhodovací strom, *náhodný les*, *gradient boosting*) pro nelineární vztahy a smíšené typy příznaku - na tabulárních datech bývají nejlepší a robustní. Stromy dělí podle *prahu* jednoho příznaku, takže jsou *invariantní k monotónní transformaci* (škálování nezmění pořadí hodnot, tedy ani štěpení) - proto nepotřebují normalizaci.

Předzpracování: one-hot kategorie, škáluj spojité, imputuj chybějící. $MAE = \frac{1}{n} \sum |y - \hat{y}|$ (robustní), $RMSE = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$ (trestá velké chyby). Stromy/boosting bývají nejlepší a nepotřebují škálování.

Kalibrace: Tabulární workflow (10%) se objevil nedávno a chtěl předzpracování + metriky (ne jen definici modelu). $MAE/RMSE$ a one-hot/škálování/imputace = 2-bodová záloha proti „end-to-end“ otázce.

Pozor (častá chyba): One-hot pro neuspořádané kategorie (jinak modelu podsouváš falešné pořadí). $RMSE$ je v jednotkách cíle (odmocnina), MSE ne; stromy škálování nepotřebují, lineární/kNN/SVM ano.

4 Specializace: Základy umělé inteligence (Téma 1)

Úloha 30. specializace UI-SU Splňování podmínek (CSP), hranová konzistence, AC-3

- (a) **1 b** Definujte problém splňování podmínek (CSP) a pojem *hranové konzistence* (arc consistency).
- (b) **2 b** Popište algoritmus AC-3 (fronta hran, revize domény, šíření) a uveďte jeho časovou složitost.
- (c) **3 b** Pro proměnné X, Y, Z s doménami $\{1, 2, 3\}$ a omezeními $X < Y, Y < Z$ proveďte AC-3 a uveďte výsledné domény. Vysvětlete rozdíl mezi lokální (hranovou) a globální konzistencí a proč hranová konzistence nezaručí existenci řešení.

(a) **Definice.** CSP je trojice $(\mathcal{V}, \mathcal{D}, \mathcal{C})$: proměnné $\mathcal{V} = \{X_1, \dots, X_n\}$, jejich domény $\mathcal{D} = \{D_1, \dots, D_n\}$ a omezení $\mathcal{C}$ (každé omezuje přípustné kombinace hodnot podmnožiny proměnných). Řešení = přiřazení hodnot z domén splňující všechna omezení. Hrana (X_i, X_j) (binární omezení) je *hranově konzistentní*, pokud pro každou hodnotu $a \in D_i$ existuje hodnota $b \in D_j$ taková, že dvojice (a, b) splňuje omezení. CSP je hranově konzistentní, jsou-li konzistentní všechny hrany.

(b) **AC-3.** Udržuj frontu všech hran (orientovaně, obě strany každého omezení). Opakovaně vyjmi hranu (X_i, X_j) a proveď REVISE: odstraň z D_i každou hodnotu, pro kterou v D_j neexistuje podpora. Pokud se D_i zmenšila, vlož zpět do fronty všechny hrany (X_k, X_i) pro sousedy X_k (mohla se porušit jejich konzistence). Konec, když je fronta prázdná, nebo nějaká doména zůstane prázdná (pak CSP nemá řešení). Složitost: $O(e d^3)$, kde e je počet hran (binárních omezení) a d velikost domény (každá z $O(e)$ hran se zařadí nejvýše d -krát, jedna REVISE je $O(d^2)$).

(c) **Průběh.** Z $X < Y$: X nemůže být 3 (nic většího v Y) $\Rightarrow D_X = \{1, 2\}$; Y nemůže být 1 $\Rightarrow D_Y = \{2, 3\}$. Z $Y < Z$: Y nemůže být 3 $\Rightarrow D_Y = \{2\}$; Z nemůže být 1 ani 2 $\Rightarrow D_Z = \{3\}$. Zmenšení D_Y vrátí do fronty hranu (X, Y) : X nyní potřebuje $y > x$ s $y \in \{2\}$, takže $x = 2$ vypadne $\Rightarrow D_X = \{1\}$. Výsledek: $D_X = \{1\}, D_Y = \{2\}, D_Z = \{3\}$ (zde jediné řešení).

Lokální vs. globální. Hranová konzistence je *lokální* vlastnost (po dvojicích). Nezaručuje globální řešení: např. tři proměnné s doménami $\{1, 2\}$ a omezeními „navzájem různé” jsou hranově konzistentní (každá hodnota má podporu u souseda), ale tři po dvou různé hodnoty z dvouprvkové domény neexistují, takže CSP je nesplnitelný. Proto se AC-3 kombinuje s prohledáváním (backtracking), typicky algoritmus MAC = po každém přiřazení znovu udělat hranovou konzistenci.

CSP = (V, D, C) . Hranová konzistence: každá hodnota $a \in D_i$ má podporu $b \in D_j$ splňující omezení. AC-3 (fronta hran, revize, šíření při zúžení), $O(e d^3)$. Nezaručuje řešení $\rightarrow$ nutný backtracking/MAC.

Kalibrace: Specializaci tvoří 3 ML + 1 „Základy UI”. AI okruhy (CSP, minimax, Bayesovské sítě, A*, MDP, HMM) jsou jednotlivě vzácné, ale slot na jednu z nich je skoro vždy. Proto v úrovni 1 musí mít každá AI rodina aspoň 2-bodovou zálohu: přesná definice + jeden kanonický postup.

Pozor (častá chyba): Hranová konzistence *nedokazuje* řešitelnost - to je oblíbená chytací podotázka. Vždy zmiň protipříklad (např. all-different na malé doméně) a roli backtrackingu/MAC.

Úloha 31. specializace UI-SU Prohledávání: neinformované a informované (A*)

- (a) **1 b** Popište formulaci úlohy prohledáváním (stav, operátory, cíl, cena). Rozdíl mezi stromovým a grafovým prohledáváním. Uveďte neinformované metody BFS, DFS a uniform-cost.
- (b) **2 b** Popište algoritmus A* a funkci $f(n) = g(n) + h(n)$. Co je přípustná a co konzistentní heuristika?

- (c) **3 b** Za jakých podmínek je A^* optimální? Co znamená dominance heuristik a proč je lepší „větší“ přípustná heuristika?

(a) *Formulace*: počáteční stav, množina *operátoru* (akce s cenou) generujících následníky, *cílový test*, cena cesty. *Stromové* prohledávání neviduje navštívené stavy (může opakovat), *grafově* drží uzavřenou množinu (closed set). *BFS*: po vrstvách (fronta), úplné, optimální při jednotkových cenách. *DFS*: do hloubky (zásobník), paměťově levné, neúplné/nesuboptimální. *Uniform-cost*: rozšiřuje uzel s nejmenší g (Dijkstra), optimální pro nezáporné ceny.

(b) A^* rozšiřuje uzel s nejmenším $f(n) = g(n) + h(n)$, kde g je cena z počátku a h *heuristický* odhad ceny do cíle. Heuristika je *přípustná*, pokud nikdy nepřeceňuje ($0 \leq h(n) \leq h^*(n)$); *konzistentní* (monotónní), pokud $h(n) \leq c(n, n') + h(n')$ pro každý přechod (trojúhelníková nerovnost).

(c) A^* je optimální se *přípustnou* heuristikou ve stromovém prohledávání a s *konzistentní* heuristikou v grafovém (jinak je nutné znovuvytváření uzlu). *Dominance*: h_2 dominuje h_1 , je-li $h_2(n) \geq h_1(n)$ pro všechny n (a obě přípustné); pak A^* s h_2 rozšíří méně uzlu (efektivnější), protože vyšší přípustný odhad ořeže víc.

A^* rozšiřuje uzel s nejmenším $f(n) = g(n) + h(n)$. Přípustná: $h \leq h^*$ (nepřeceňuje). Konzistentní: $h(n) \leq c(n, n') + h(n')$. Optimální s přípustnou (stromově) / konzistentní (grafově) heuristikou.

Kalibrace: Prohledávání (15%) je opakovaná „Základy UI“ otázka (A^* se objevil). Definice A^* + přípustnost/konzistence = 2-bodová záloha pro AI slot specializace.

Pozor (častá chyba): Přípustná $\neq$ konzistentní (konzistence je silnější a implikuje přípustnost). V grafovém A^* je pro optimalitu potřeba konzistence, ne jen přípustnost.

Úloha 32. specializace UI-SU Hry: minimax, alfa-beta a teorie her

- (a) **1 b** Popište algoritmus Minimax pro hru dvou hráčů s nulovým součtem. Co předpokládá o protihráči?
- (b) **2 b** Popište alfa-beta prořezávání: co a proč prořezává a jak pořadí tahu ovlivní úsporu.
- (c) **3 b** Vysvětlete základy teorie her: věžňovo dilema, dominantní strategie, Nashovo ekvilibrium a Paretovskou optimalitu. Co je mechanism design a jaké jsou typy aukcí?

(a) *Minimax*: prohledá herní strom; hráč MAX volí tah maximalizující hodnotu, MIN minimalizující. Hodnota listu = užitek; hodnota vnitřního uzlu = max/min hodnot synu podle toho, kdo je na tahu. Předpokládá *optimálně hrajícího* protihráče (nejhorší případ pro nás). Hloubku lze omezit a listy nahradit *ohodnocovací funkcí*.

(b) *Alfa-beta* udržuje α (nejlepší jistá hodnota pro MAX) a β (pro MIN). Větev se *ořízne*, jakmile $\alpha \geq \beta$, protože ji racionální hráč stejně nezvolí, takže nemá smysl ji dál prohledávat. Výsledek je identický s minimaxem. Při *dobrém pořadí* tahu (nejlepší první) klesne počet prohledaných uzlu z $O(b^d)$ až na $O(b^{d/2})$, tj. dvojnásobná dosažitelná hloubka.

(c) *Věžňovo dilema*: dva hráči, každý volí spolupráci/zradu; zrada je *dominantní strategie* (lepší bez ohledu na soupeře), takže oba zradí, ač by oboustranná spolupráce byla lepší. *Nashovo ekvilibrium*: profil strategií, kde nikdo nezíská jednostrannou změnou (vzájemně nejlepší odpovědi); ve věžňově dilematu je to (zrada, zrada). *Paretoovsky optimální* je výsledek, který nelze zlepšit jednomu bez zhoršení druhému - (spolupráce, spolupráce) je Pareto-lepší, ale není Nashovo ekvilibrium. *Mechanism design* je „obrácená“ teorie her: navrhuje pravidla

hry tak, aby racionální hráči svým chováním dosáhli žádoucího výsledku. Příklad jsou *aukce*: anglická (rostoucí, otevřená), holandská (klesající), prvocenová obálková a *Vickreyova* (druhá cena) - u Vickreyovy je dominantní strategií dražit svou pravou hodnotu (pravdomlnost).

Minimax: MAX maximalizuje, MIN minimalizuje hodnoty listu. Alfa-beta ořeže větev při $\alpha \geq \beta$ (až $O(b^{d/2})$). Nashovo ekvilibrium = profil, kde nikdo nezíská jednostrannou změnou (věžňovo dilema: (zrada, zrada)).

Kalibrace: Hry (22 %, minimax + teorie her) jsou opakovaná „Základy UI“ otázka. Minimax + alfa-beta + Nash = 2-3 body za vysvětlení.

Pozor (častá chyba): Alfa-beta nemění *výsledek*, jen rychlost. Nashovo ekvilibrium nemusí být Paretovsky optimální (věžňovo dilema je důkaz).

Úloha 33. specializace UI-SU Bayesovské sítě a pravděpodobnostní inference

- (a) **1 b** Definujte Bayesovskou síť a její vztah k úplné sdružené distribuci (faktorizace).
- (b) **2 b** Jak se síť konstruuje (podmíněná nezávislost) a jak se počítá exaktní inference enumerací?
- (c) **3 b** Popište eliminaci proměnných a aproximační inferenci (Monte Carlo: zamítání, vážení věrohodností).

(a) *Bayesovská síť* = orientovaný acyklický graf, kde uzly jsou náhodné proměnné a hrany vyjadřují přímou závislost; každý uzel má tabulku podmíněných pravděpodobností $P(X_i \mid \text{rodiče}(X_i))$. Kóduje *úplnou sdruženou distribuci* faktorizací

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{rodiče}(X_i)),$$

což šetří parametry oproti plné tabulce (využívá podmíněné nezávislosti).

(b) *Konstrukce*: seřaď proměnné (ideálně příčiny před následky) a každou napoj jen na ty předchozí, na nichž přímo závisí (Markovský rodičovský obal). *Inference enumerací* dotazu $P(Q \mid e)$: $P(Q \mid e) \propto \sum_{\text{skryté } h} P(Q, e, h)$, kde sčítáme součiny faktoru z faktorizace přes všechny hodnoty skrytých proměnných a nakonec normujeme.

(c) *Eliminace proměnných* sčítá zprava efektivněji: postupně „vysčítá“ skryté proměnné, mezivýsledky drží jako faktory a opakované součiny nepře počítává - výrazně rychlejší než holá enumerace. *Aproximace (Monte Carlo): zamítavé vzorkování* (rejection) generuje vzorky z priorů a zahodí ty neslučitelné s e (neefektivní při vzácném e); *vážení věrohodností* (likelihood weighting) fixuje pozorované proměnné a každý vzorek váží součinem jejich pravděpodobností - žádný vzorek nezahazuje.

Bayesovská síť = DAG + tabulky $P(X_i \mid \text{rodiče}(X_i))$, kóduje $P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{rodiče}(X_i))$. Inference: $P(Q \mid e) \propto \sum_{\text{skryté } h} P(Q, e, h)$ (enumerace, efektivněji eliminace proměnných).

Kalibrace: Bayesovské sítě (22 % s Bayesovským učením) jsou opakovaná „Základy UI“ otázka. Faktorizace + enumerace = 2-bodová záloha; tady je AI rodina, kde je vhodné mít i hlubší zvládnutí.

Pozor (častá chyba): Faktorizace je součin $P(X_i \mid \text{rodiče})$ - ne marginálu. Likelihood weighting nezahazuje vzorky (na rozdíl od rejection), proto je efektivnější při vzácné evidenci.

Úloha 34. specializace UI-SU Bayesovské učení a EM algoritmus

- (a) **1 b** Popište Bayesovské učení: prior, věrohodnost, posterior. Co je maximálně věrohodná (ML) a maximálně aposteriorní (MAP) hypotéza a co je Bayesovsky optimální predikce?
- (b) **2 b** Popište EM algoritmus (E-krok a M-krok) pro model se skrytou proměnnou.
- (c) **3 b** Napište vzorec pro responsibility (odpovědnost) v E-kroku a aktualizaci parametru v M-kroku pro skrytou kategoriální proměnnou (shluk).

- (a) Bayesovo pravidlo: $P(h | D) = \frac{P(D|h)P(h)}{P(D)}$, kde $P(h)$ je *prior*, $P(D | h)$ *věrohodnost*, $P(h | D)$ *posterior*. *ML hypotéza* $h_{ML} = \arg \max_h P(D | h)$; *MAP* $h_{MAP} = \arg \max_h P(D | h)P(h)$ (= ML při uniformním prioru). *Bayesovsky optimální predikce* nepoužívá jedinou hypotézu, ale váží přes všechny: $P(c | D) = \sum_h P(c | h)P(h | D)$.
- (b) *EM* maximalizuje věrohodnost při skrytých proměnných střídáním: *E-krok* spočítá očekávané rozdělení skrytých proměnných (responsibility) za daných aktuálních parametru; *M-krok* přepočítá parametry tak, aby maximalizovaly očekávanou (úplnou) log-věrohodnost. Iteruje do konvergence; věrohodnost monotónně neklesá, konverguje do lokálního maxima.
- (c) Pro skrytý shluk $z \in \{1, \dots, K\}$ s vahami (priors) π_k a modelem $p(x | \theta_k)$ je *responsibility* bodu x_i vůči shluku k :

$$r_{ik} = P(z_i = k | x_i) = \frac{\pi_k p(x_i | \theta_k)}{\sum_{j=1}^K \pi_j p(x_i | \theta_j)}.$$

M-krok: $\pi_k = \frac{1}{n} \sum_i r_{ik}$ a parametry θ_k se odhadnou váženě responsibilitymi (např. vážený průměr $\mu_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}$); u kategoriálních dat = vážené relativní četnosti.

$h_{MAP} = \arg \max_h P(D | h)P(h)$ (= ML při uniformním prioru). EM: E-krok responsibility $r_{ik} = \frac{\pi_k p(x_i | \theta_k)}{\sum_j \pi_j p(x_i | \theta_j)}$; M-krok $\pi_k = \frac{1}{n} \sum_i r_{ik}$ a vážené přepočty parametru.

Kalibrace: „Bayesovské učení“ a EM jsou opakovaný „Základy UI“ okruh; EM zkouška chtěla jeden výpočetní update (*responsibility*), ne jen pojem. Vzorec responsibility umět zapaměti.

Pozor (častá chyba): MAP $\neq$ ML, liší se priorem. EM najde jen *lokální* maximum (závisí na inicializaci); responsibility se musí normovat přes všechny shluky.

Úloha 35. specializace UI-SU Markovské modely a skryté Markovské modely (HMM)

- (a) **1 b** Co je Markovský řetězec a Markovský předpoklad? Definujte skrytý Markovský model (HMM): skryté stavy, přechody, emise.
- (b) **2 b** Popište filtraci, predikci a vyhlazování a uveďte rekurentní vzorec pro filtraci (forward).
- (c) **3 b** Co počítá Viterbiho algoritmus (nejpravděpodobnější průchod)? Jak HMM souvisí s dynamickými Bayesovskými sítěmi?

(a) *Markovský řetězec*: posloupnost stavu, kde budoucnost závisí jen na současném stavu (*Markovský předpoklad* $P(X_t | X_{1:t-1}) = P(X_t | X_{t-1})$). *HMM*: skryté stavy X_t s maticí přechodu $P(X_t | X_{t-1})$ a pozorování E_t s modelem emisí $P(E_t | X_t)$; vidíme jen E_t , stavy odhadujeme.

(b) *Filtrace*: $P(X_t \mid e_{1:t})$ (odhad současného stavu z dosavadních pozorování). *Predikce*: $P(X_{t+k} \mid e_{1:t})$ (budoucí stav). *Vyhlašování*: $P(X_k \mid e_{1:t})$ pro $k < t$ (minulý stav s využitím pozdějších pozorování). Rekurence filtrace (forward):

$$P(X_t \mid e_{1:t}) \propto P(e_t \mid X_t) \sum_{x_{t-1}} P(X_t \mid x_{t-1}) P(x_{t-1} \mid e_{1:t-1}),$$

tj. krok predikce (přes přechod) a poté přenásobení emisí a normalizace.

(c) *Viterbi* dynamickým programováním najde *nejpravděpodobnější posloupnost skrytých stavů* pro dané pozorování (max místo součtu ve forward rekurzi + zpětný chod). HMM je speciální případ *dynamické Bayesovské sítě* (řetězec s jednou skrytou proměnnou na krok); DBN dovolují víc proměnných a strukturovanějších závislostí na časový krok.

HMM = skryté stavy (přechody $P(X_t \mid X_{t-1})$) + emise $P(E_t \mid X_t)$. Filtrace (forward): $P(X_t \mid e_{1:t}) \propto P(e_t \mid X_t) \sum_{x_{t-1}} P(X_t \mid x_{t-1}) P(x_{t-1} \mid e_{1:t-1})$. Viterbi = nejpravděpodobnější cesta (max místo sumy).

Kalibrace: HMM (12%) je opakované „Základy UI“ téma (filtrace se zkoušela numericky). Definice + forward rekurence = 2-bodová záloha; pro 3 body umět jeden krok filtrace dosadit.

Pozor (častá chyba): Filtrace je jen „dopředu“, vyhlazování využívá i *budoucí* pozorování. Viterbi hledá nejpravděpodobnější *celou cestu*, ne stav po stavu.

Úloha 36. specializace UI-SU Markovské rozhodovací procesy (MDP) a zpětnovazební učení

- (a) **1b** Definujte MDP (stavy, akce, přechody, odměny, diskont) a pojmy strategie (policy) a užitek (hodnota) stavu.
- (b) **2b** Napište Bellmanovu rovnici pro optimální hodnotu a popište iteraci hodnot (value iteration).
- (c) **3b** Co je iterace strategií? Popište zpětnovazební učení: Q-učení (update) a kompromis exploração vs exploítace.

(a) $MDP = (S, A, P, R, \gamma)$: stavy S , akce A , přechody $P(s' \mid s, a)$, odměny $R(s, a, s')$, diskont $\gamma \in [0, 1]$. *Strategie* $\pi : S \rightarrow A$ určuje akci v každém stavu. *Hodnota* (užitek) stavu při π : očekávaná diskontovaná suma odměn $V^\pi(s) = \mathbb{E}[\sum_{t \geq 0} \gamma^t R_t \mid s_0 = s, \pi]$.

(b) *Bellmanova rovnice* pro optimální hodnotu:

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V^*(s')].$$

Iterace hodnot: inicializuj V , opakovaně aplikuj pravou stranu jako přiřazení (Bellmanuv update) pro všechny stavy, dokud se V nestabilizuje; pak optimální strategie $\pi^*(s) = \arg \max_a \sum_{s'} P[R + \gamma V^*]$.

(c) *Iterace strategií*: střídá *vyhodnocení* dané strategie (vyřeš V^π) a *zlepšení* (greedy vůči V^π); konverguje v konečně krocích. *Zpětnovazební učení* (neznámé P, R): *Q-učení* aktualizuje akční hodnotu z pozorovaného přechodu

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

(off-policy). *Explorace vs exploítace*: je třeba občas zkusit i nepreferované akce (např. ϵ -greedy), jinak agent uvázne v suboptimu; SARSA je on-policy obdoba (místo max použije skutečně

zvolenou a'). *POMDP* (částečně pozorovatelný MDP): agent stav přímo nevidí, udržuje *belief* (rozdělení pravděpodobnosti nad stavy), které po akci a pozorování aktualizuje (Bayes), a rozhoduje se podle něj.

MDP = (S, A, P, R, γ) . Bellman: $V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R + \gamma V^*(s')]$; iterace hodnot aplikuje toto jako update. Q-učení: $Q(s, a) \leftarrow Q + \alpha [r + \gamma \max_{a'} Q(s', a') - Q]$.

Kalibrace: MDP/RL (15 %) je opakovaná „Základy UI“ otázka (Bellmanova rovnice se zkoušela). Definice MDP + Bellman + iterace hodnot = 2-bodová záloha.

Pozor (častá chyba): Bellman pro optimum má $\max_a$ (u dané strategie je tam suma přes π). Q-učení je off-policy (max), SARSA on-policy - nezaměňuj.

Úloha 37. specializace UI-SU Logické uvažování: DPLL, řetězení, rezoluce

- (a) **1 b** Zopakujte CNF a popište algoritmus DPLL pro řešení splnitelnosti (SAT): jednotková propagace a čistý literál.
- (b) **2 b** Co jsou Hornovy klauzule a jak funguje dopředné a zpětné řetězení?
- (c) **3 b** Popište rezoluci jako rozhodovací/dokazovací metodu (důkaz sporem, prázdná klauzule). K čemu je v predikátové logice unifikace?

(a) *CNF* = konjunkce klauzulí (disjunkcí literálů). *DPLL* prohledává přiřazení s prořezáváním: (1) *jednotková propagace* - klauzule s jediným literálem vynutí jeho hodnotu (a zjednoduší ostatní); (2) *čistý literál* - proměnná vyskytující se jen v jedné polaritě se nastaví tak, aby tyto klauzule splnila; (3) jinak rozděl podle nějaké proměnné (větev true/false) a rekurzivně. Konec: prázdná množina klauzulí = splnitelné, prázdná klauzule = konflikt (backtrack).

(b) *Hornova klauzule* má nejvýše jeden pozitivní literál (tj. tvar $a_1 \wedge \dots \wedge a_k \Rightarrow b$). *Dopředné řetězení*: z známých faktů opakovaně odvozuj hlavy pravidel, jejichž těla jsou splněna, dokud nepřibývají nové fakty (data-driven). *Zpětné řetězení*: od dotazovaného cíle rekurzivně dokazuj těla pravidel, která ho mají v hlavě (goal-driven, základ Prologu). Pro Hornovy klauzule je to lineární/efektivní.

(c) *Rezoluce*: dokazuje $T \models \varphi$ sporem - k T přidá $\neg\varphi$, převede na CNF a opakovaně tvoří rezolventy; odvození *prázdné klauzule* znamená nespílitelnost, tedy platnost φ . V *predikátové logice* je nutná *unifikace*: najítí nejobecnějšího substitučního přiřazení proměnných, které dva literály ztotožní, aby šly rezolvovat.

DPLL = jednotková propagace + čistý literál + větvení s backtrackingem. Hornovy klauzule (≤ 1 pozitivní literál) = dopředné/zpětné řetězení. Rezoluce = důkaz sporem (přidej $\neg\varphi$, odvoď prázdnou klauzuli); v predikátové logice s unifikací.

Kalibrace: Logické uvažování (10 %, DPLL se zkoušelo) je vzácnější „Základy UI“ okruh. CNF + DPLL kroky = 2-bodová záloha; navíc se prolíná s matematickou logikou.

Pozor (častá chyba): Jednotková propagace a čistý literál nejsou totéž. Rezoluce dokazuje sporem (přidej negaci cíle); cílem je odvodit prázdnou klauzuli.

Úloha 38. specializace UI-SU Automatické plánování a reprezentace znalostí

- (a) **1 b** Popište formulaci plánovacího problému a definici operátoru (předpoklady a efekty), počáteční stav a cíl.
- (b) **2 b** Vysvětlete dopředné (progression) a zpětné (regression) plánování a jejich rozdíl.
- (c) **3 b** Co je situační kalkulus a problém rámce (frame problem) a jak se řeší?

(a) *Plánovací problém* (např. STRIPS): počáteční stav (množina platných faktů), množina operátorů a cílová podmínka. *Operátor* má *předpoklady* (precondition - co musí platit, aby šel provést) a *efekty* (add list = co začne platit, delete list = co přestane platit). Plán = posloupnost operátorů vedoucí z počátečního stavu do stavu splňujícího cíl.

(b) *Dopředné plánování*: začni v počátečním stavu a aplikuj použitelné operátory, dokud nedosáhneš cíle (prohledávání stavového prostoru vpřed); velký větvící faktor. *Zpětné plánování*: začni od cíle a hledej operátory, jejichž efekty cíl (částečně) splňují, a nahraď cíl jejich předpoklady (regrese); pracuje jen s relevantními operátory, ale stavy jsou částečné (množiny podcílu).

(c) *Situační kalkulus* popisuje svět v predikátové logice: *fluenty* (vlastnosti závislé na situaci, např. $At(x, s)$), akce a funkci $do(a, s)$ pro situaci po akci. *Problém rámce*: je třeba vyjádřit nejen co se akcí *změní*, ale i co *zůstane beze změny* - jinak by se počet axiomů rozrostl pro každou dvojici akce/fluente. Řeší se *successor-state axiomy* (pro každý fluent jeden axiom popisující, kdy v nové situaci platí), což stručně zachytí setrvačnost (co akce neovlivní, zůstává).

Operátor = (precondition, add efekty, delete efekty). *Dopředné* plánování jde od počátku vpřed, *zpětné* regreduje od cíle přes efekty operátoru. *Problém rámce* = jak vyjádřit, co se akcí nemění (successor-state axiomy).

Kalibrace: Plánování a reprezentace znalostí jsou vzácnější „Základy UI“ rodiny, ale legální draw do skoro vždy přítomného AI slotu. Bez této zálohy hrozí na takové otázce skóre 0-1, což přímo ohrožuje podlahu specializace ($\geq 7/12$).

Pozor (častá chyba): Dopředné vs zpětné neplet: regrese jde od cíle a nahrazuje cíl předpoklady operátoru. *Problém rámce* není o „výpočtu“, ale o stručném popisu neměnnosti.